Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
GRADUATION THESIS

Разработка средства оценки планируемой нагрузки СУБД PostgreSQL на систему ввода-вывода

Обучающийся / Student Щербаков Александр Валерьевич
Факультет/институт/кластер/ Faculty/Institute/Cluster факультет программной инженерии и компьютерной техники
Группа/Group P34151
Направление подготовки/ Subject area 09.03.04 Программная инженерия
Образовательная программа / Educational program Системное и прикладное программное обеспечение 2021
Язык реализации ОП / Language of the educational program Русский
Квалификация/ Degree level Бакалавр
Руководитель ВКР/ Thesis supervisor Логинов Иван Павлович, кандидат технических наук, Университет ИТМО, факультет программной инженерии и компьютерной техники, доцент (квалификационная категория "ординарный доцент")

Обучающийся/Student

Документ подписан
Щербаков Александр Валерьевич
23.05.2025

(эл. подпись/ signature)

Щербаков Александр Валерьевич
(Фамилия И.О./ name and surname)

Руководитель ВКР/ Thesis supervisor

Документ подписан
Логинов Иван Павлович
15.05.2025

(эл. подпись/ signature)

Логинов Иван Павлович
(Фамилия И.О./ name and surname)

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ / OBJECTIVES FOR A GRADUATION THESIS

Обучающийся / Student Щербаков Александр Валерьевич
Факультет/институт/кластер/ Faculty/Institute/Cluster факультет программной инженерии и компьютерной техники
Группа/Group P34151
Направление подготовки/ Subject area 09.03.04 Программная инженерия
Образовательная программа / Educational program Системное и прикладное программное обеспечение 2021
Язык реализации ОП / Language of the educational program Русский
Квалификация/ Degree level Бакалавр
Тема ВКР/ Thesis topic Разработка средства оценки планируемой нагрузки СУБД PostgreSQL на систему ввода-вывода
Руководитель ВКР/ Thesis supervisor Логинов Иван Павлович, кандидат технических наук, Университет ИТМО, факультет программной инженерии и компьютерной техники, доцент (квалификационная категория "ординарный доцент")

Характеристика темы ВКР / Description of thesis subject (topic)

Тема в области фундаментальных исследований / Subject of fundamental research: нет / not
Тема в области прикладных исследований / Subject of applied research: да / yes

Основные вопросы, подлежащие разработке / Key issues to be analyzed
Цель работы: обеспечение возможности предварительной оценки нагрузки на подсистему ввода-вывода PostgreSQL для предотвращения деградации производительности и обеспечения стабильной работы системы при различных сценариях эксплуатации.

Задачи:
1) Проанализировать существующие методы оценки нагрузки на систему ввода-вывода в контексте работы СУБД PostgreSQL.
2) Выявить архитектурные особенности PostgreSQL, влияющие на характер нагрузки на подсистему ввода-вывода.
3) Разработать модель прогнозирования планируемой I/O-нагрузки на основе анализа метаданных и предполагаемых сценариев работы с данными.
4) Реализовать программное средство, позволяющее автоматизировать процесс оценки.
5) Провести тестирование разработанного средства на различных сценариях использования базы данных.

Рекомендуемые материалы и пособия:

- Документация PostgreSQL (https://www.postgresql.org/docs/)
- Статьи по работе PostgreSQL
- Boncz et al. "Database Architecture Optimized for the New Bottleneck: Memory Access" (2008)
- Stonebraker et al. "MapReduce and Parallel DBMSs: Friends or Foes?" (2010)

Форма представления материалов ВКР / Format(s) of thesis materials:
Основные или дополнительные результаты ВКР представляются в форме текста ВКР и презентации. Формой представления Приложений является программный код.

Дата выдачи задания / Assignment issued on: 14.10.2024

Срок представления готовой ВКР / Deadline for final edition of the thesis 25.05.2025

СОГЛАСОВАНО / AGREED:

Руководитель ВКР/ Thesis supervisor	Документ подписан Логинов Иван Павлович 05.05.2025 (эл. подпись)	Логинов Иван Павлович
Задание принял к исполнению/ Objectives assumed BY	Документ подписан Щербаков Александр Валерьевич 14.05.2025 (эл. подпись)	Щербаков Александр Валерьевич
Руководитель ОП/ Head of educational program	Документ подписан Дергачев Андрей Михайлович 18.05.2025 (эл. подпись)	Дергачев Андрей Михайлович

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

АННОТАЦИЯ
ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ
SUMMARY OF A GRADUATION THESIS

Обучающийся / Student Щербаков Александр Валерьевич
Факультет/институт/кластер/ Faculty/Institute/Cluster факультет программной инженерии и компьютерной техники
Группа/Group P34151
Направление подготовки/ Subject area 09.03.04 Программная инженерия
Образовательная программа / Educational program Системное и прикладное программное обеспечение 2021
Язык реализации ОП / Language of the educational program Русский
Квалификация/ Degree level Бакалавр
Тема ВКР/ Thesis topic Разработка средства оценки планируемой нагрузки СУБД PostgreSQL на систему ввода-вывода
Руководитель ВКР/ Thesis supervisor Логинов Иван Павлович, кандидат технических наук, Университет ИТМО, факультет программной инженерии и компьютерной техники, доцент (квалификационная категория "ординарный доцент")

ХАРАКТЕРИСТИКА ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ
DESCRIPTION OF THE GRADUATION THESIS

Цель исследования / Research goal
Обеспечение возможности предварительной оценки нагрузки на подсистему ввода-вывода PostgreSQL для предотвращения деградации производительности и обеспечения стабильной работы системы при различных сценариях эксплуатации.
Задачи, решаемые в ВКР / Research tasks
1) Проанализировать существующие методы оценки нагрузки на систему ввода-вывода в контексте работы СУБД PostgreSQL. 2) Выявить архитектурные особенности PostgreSQL, влияющие на характер нагрузки на подсистему ввода-вывода. 3) Разработать модель прогнозирования планируемой I/O-нагрузки на основе анализа метаданных и предполагаемых сценариев работы с данными. 4) Реализовать программное средство, позволяющее автоматизировать процесс оценки. 5) Провести тестирование разработанного средства на различных сценариях использования базы данных.
Краткая характеристика полученных результатов / Short summary of results/findings
Были проанализированы существующие методы оценки I/O-нагрузки в контексте СУБД PostgreSQL, выявлены архитектурные особенности, определяющие характер взаимодействия с подсистемой ввода-вывода. Разработана модель прогнозирования предполагаемой нагрузки на основе анализа метаданных и сценариев работы с данными. Реализовано программное средство, автоматизирующее процесс оценки, и проведено его тестирование на различных практических сценариях, что подтвердило корректность и

применимость разработанного подхода.

Обучающийся/Student

Документ подписан
Щербаков Александр Валерьевич
23.05.2025

(эл. подпись/ signature)

Щербаков Александр Валерьевич

(Фамилия И.О./ name and surname)

Руководитель ВКР/
Thesis supervisor

Документ подписан
Логинов Иван Павлович
15.05.2025

(эл. подпись/ signature)

Логинов Иван Павлович

(Фамилия И.О./ name and surname)

СОДЕРЖАНИЕ

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ . . 9

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ 10

ВВЕДЕНИЕ 11

1 Анализ предметной области и существующих решений 14

1.1 Проблематика оценки нагрузки на систему ввода-вывода в СУБД 14

1.2 Способы и средства оценки нагрузки на диск 15

1.2.1 Профилирование реальной нагрузки 15

1.2.2 Моделирование нагрузки на основе статистики . 15

1.2.3 Теоретическое моделирование 16

1.2.4 Коммерческие решения 16

1.2.5 Использование методов машинного обучения . . 17

1.3 Промежуточные выводы 19

1.3.1 Сравнительная таблица существующих решений 20

2 Особенности архитектуры PostgreSQL, влияющие на I/O-нагрузку 21

2.1 Параметры конфигурации влияющие на нагрузку на диск 22

2.2 Структура хранения данных в PostgreSQL 23

2.2.1 Таблицы и файлы: связь объектов БД с файлами 23

2.2.2 Каталожные таблицы и их физическое представление 23

2.2.3 Сегментация файлов (segmenting) больших объектов 23

2.2.4 Механизм TOAST и хранение больших данных . 24

2.2.5 Индексы и их файловая структура 24

2.3 Журналирование и Write-Ahead Logging (WAL) в PostgreSQL 24

2.3.1 Принцип Write-Ahead Logging 25

2.3.2 Роль WAL в оценке нагрузки и производительности 25

3 Проектирование и разработка средства оценки планируемой нагрузки на систему ввода-вывода PostgreSQL . 27
3.1 Постановка требований к разрабатываемому средству . 27
3.2 Разработка программного средства 27
3.2.1 Выбор инструментов и технологий 27
3.2.2 Описание алгоритма прогнозирования 31
3.2.3 Формат входных данных основного алгоритма . 33
3.2.4 Формат выходных данных алгоритма предсказания нагрузки на систему ввода-вывода 38
3.3 Реализация web-интерфейса 40
3.3.1 Основные требования к web-интерфейсу 40
3.3.2 Выбор инструментов реализации 40
3.3.3 Архитектура и компоненты интерфейса 41
3.3.4 Ввод информации о таблицах 42
3.3.5 Панель настройки параметров PostgreSQL 43
3.3.6 Ввод SQL-запроса и интенсивности работы . . . 43
3.3.7 Валидация и обработка введённых данных 43
3.3.8 Взаимодействие с серверной частью 44
3.3.9 Резюме по реализации интерфейса 44
3.4 Автоматизация запуска с помощью Bash и интеграция с GitHub Actions 44
3.5 Резюме по реализации программного средства 47

4 Тестирование и апробация разработанного решения . . 49
4.1 Методика апробации и валидирования разрабатываемого средства 49
4.2 Описание тестовых примеров и сценариев 50
4.3 Тестовая среда и ресурсы 50
4.3.1 Определение характеристик устройства хранения данных 53
4.4 Сравнение результатов предсказания с реальными замерами 56
4.5 Анализ точности предсказаний 61
4.5.1 Методика приведения расчётных значений к фактическим ограничениям 62

4.6 Анализ ошибок и отклонений 63
4.7 Обсуждение ограничений текущей реализации 64
4.8 Выводы по результатам апробации 64

ЗАКЛЮЧЕНИЕ 66

ПРИЛОЖЕНИЕ А. Настройки тестовой среды 75

ПРИЛОЖЕНИЕ Б. Конфигурация для экспериментов 79
Б.1 `example1.yaml` 79
Б.2 `example2.yaml` 80
Б.3 `example3.yaml` 81
Б.4 `example4.yaml` 81

ПРИЛОЖЕНИЕ В. Часть алгоритма прогнозирования 82

ПРИЛОЖЕНИЕ Г. Репозиторий с исходным кодом проекта . 85

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

Сокраще-ние	**Расшифровка**
СУБД	Система управления базами данных
I/O	Input/Output (ввод-вывод)
SQL	Structured Query Language (язык структурированных запросов)
TPS	Transactions Per Second (транзакций в секунду)
SSD	Solid-State Drive (накопитель на твёрдотельной памяти)
HDD	Hard Disk Drive (жёсткий диск)
CPU	Central Processing Unit (центральный процессор)
RAM	Random Access Memory (оперативная память)
WAL	Write-Ahead Logging (журнал предзаписи в PostgreSQL)
DBMS	Database Management System (англ. эквивалент СУБД)
IOPS	Input/Output Operations Per Second (операций ввода-вывода в секунду)
RPS	Requests Per Second (запросов в секунду)
TOAST	The Oversized-Attribute Storage Technique (техника хранения переразмеренных атрибутов в PostgreSQL)

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

Система управления базами данных (СУБД) -— программное обеспечение, обеспечивающее создание, модификацию, управление и доступ к базам данных.

Подсистема ввода-вывода (I/O-система) —- часть вычислительной системы, обеспечивающая обмен данными между оперативной памятью и устройствами хранения данных (дисками, SSD и др.).

Нагрузка на систему ввода-вывода —- совокупность операций чтения и записи данных, выполняемых в процессе работы приложения или СУБД.

Планируемая нагрузка —- ожидаемое количество операций ввода-вывода, которое должно быть выполнено системой на основании проектных характеристик приложений и базы данных.

Производительность СУБД —- способность системы управления базами данных обрабатывать определённый объём транзакций или запросов за единицу времени при заданных ресурсах.

Запрос -— команда, направленная к СУБД для получения, изменения или удаления данных.

Профилирование нагрузки -— процесс измерения характеристик работы системы для последующего анализа производительности и поиска узких мест.

Тестирование производительности -— выполнение заранее определённого набора операций для оценки скорости работы системы при различных условиях нагрузки.

Модель нагрузки —- абстрактное представление сценариев взаимодействия пользователя или приложения с базой данных, описывающее частоту и характер операций.

ВВЕДЕНИЕ

В современных информационных системах базы данных занимают центральное место, обеспечивая хранение, обработку и предоставление информации. При проектировании и эксплуатации систем управления базами данных (СУБД) одним из критически важных факторов является производительность. Производительность, в свою очередь, во многом определяется эффективностью работы системы ввода-вывода (I/O). [1]

Система ввода-вывода отвечает за взаимодействие между оперативной памятью и долговременными устройствами хранения данных. Несмотря на развитие технологий хранения, таких как твердотельные накопители (SSD) и системы хранения в оперативной памяти (In-Memory Databases), проблема скорости и надёжности операций чтения и записи остаётся крайне актуальной. [2]

Особенно важной задача оценки нагрузки на подсистему ввода-вывода становится на этапах проектирования новых систем или масштабирования существующих. Недооценка этой нагрузки может привести к критическим сбоям в работе приложений, увеличению времени отклика, потере данных и финансовым убыткам. [3] [4]

На практике оценка предполагаемой нагрузки часто проводится эмпирически либо на основе опыта специалистов, что не всегда позволяет достичь необходимой точности. Использование существующих инструментов мониторинга, таких как *pgbench*, *HammerDB* или системные средства статистики, также не решает проблему оценки именно планируемой, а не фактической нагрузки. [5; 6]

Современные информационные системы испытывают возрастающие нагрузки, связанные с интенсивным доступом к данным и необходимостью оптимальной организации хранения и обработки информации. В среде управления базами данных, таких как PostgreSQL, одним из критически важных аспектов производительности является нагрузка на дисковую подсистему, возникающая под воздействием различных типов запросов и операций ввода-вывода. На рынке труда ИТ-специалистов сейчас особенно высоко ценится умение масштаби-

ровать системы, что говорит об актуальности проблемы [7] Традиционные методы мониторинга и анализа нагрузки опираются на специализированные инструменты и метрики (например, `iostat`, `pg_stat_io`), однако их использование зачастую ограничено невозможностью адекватно предсказывать поведение системы в условиях изменяющихся рабочих нагрузок и аномальных ситуаций.

Таким образом, существует явная потребность в разработке средств, которые позволяли бы прогнозировать нагрузку на подсистему ввода-вывода PostgreSQL на основе анализа структуры базы данных, предполагаемых сценариев её использования и особенностей функционирования самой СУБД.

Цель работы — разработка средства оценки планируемой нагрузки на систему ввода-вывода СУБД PostgreSQL.

Для достижения поставленной цели необходимо решить следующие задачи:

- Проанализировать существующие методы оценки нагрузки на систему ввода-вывода в контексте работы СУБД PostgreSQL.
- Выявить архитектурные особенности PostgreSQL, влияющие на характер нагрузки на подсистему ввода-вывода.
- Разработать модель прогнозирования планируемой I/O-нагрузки на основе анализа метаданных и предполагаемых сценариев работы с данными.
- Реализовать программное средство, позволяющее автоматизировать процесс оценки.
- Провести тестирование разработанного средства на различных сценариях использования базы данных.

Объект исследования: процессы взаимодействия СУБД PostgreSQL с подсистемой ввода-вывода.

Предмет исследования: методы и средства прогнозирования планируемой нагрузки на систему ввода-вывода PostgreSQL.

Актуальность темы обусловлена необходимостью повышения надёжности проектируемых информационных систем, оптимизации их производительности, а также минимизации затрат на серверное оборудование за счёт более точного планирования ресурсов.

В ходе работы будут рассмотрены как существующие подходы к мониторингу и оценке нагрузки, так и предложены новые методы прогнозирования на основе анализа метаданных PostgreSQL и особенностей предполагаемой нагрузки.

1 Анализ предметной области и существующих решений

1.1 Проблематика оценки нагрузки на систему ввода-вывода в СУБД

В современных информационных системах производительность базы данных является критическим фактором, влияющим на общее качество работы приложений. Одной из ключевых составляющих производительности является взаимодействие СУБД с подсистемой ввода-вывода. [5]

Системы ввода-вывода традиционно считаются одним из узких мест в архитектуре баз данных. В отличие от операций, выполняемых исключительно в оперативной памяти, операции чтения и записи данных на дисковые устройства связаны с гораздо большими задержками. Даже несмотря на распространение быстродействующих накопителей на базе твердотельной памяти (SSD), характер нагрузки на I/O остаётся важнейшим параметром для оценки производительности. [8; 9]

Особую сложность представляет оценка планируемой нагрузки на систему ввода-вывода на ранних этапах проектирования систем. На этой стадии ещё отсутствуют реальные данные эксплуатации, а экспериментальное моделирование может быть слишком затратным по времени и ресурсам.

В ряде случаев специалисты вынуждены идти на различные ухищрения: использовать сложные схемы оркестрации и автоматизации, чтобы максимально эффективно распределять нагрузку, а также разрабатывать специальные подходы по обходу инфраструктурных ограничений.[10; 11] Это усложняет инфраструктуру, но зачастую является единственным способом обеспечить требуемый уровень производительности в условиях ограниченных I/O-ресурсов. [12]

Таким образом, необходимость в эффективных средствах предварительной оценки нагрузки на подсистему ввода-вывода в системах, использующих СУБД PostgreSQL, представляется очевидной и обоснованной.

1.2 Способы и средства оценки нагрузки на диск

1.2.1. Профилирование реальной нагрузки

Метод основан на эмпирическом тестировании, когда через генерацию типичных сценариев работы СУБД (с использованием, например, *pgbench* или *HammerDB*) непосредственно измеряется реакция системы на заданную нагрузку. Такой подход позволяет получить конкретные показатели производительности, выявить узкие места и определить, как изменения параметров конфигурации влияют на итоговую производительность базы. Он особенно полезен для оценки критических участков системы и построения сценариев оптимизации, однако его применение обременено необходимостью иметь готовую и соответствующим образом настроенную инфраструктуру, а также требует значительных временных и вычислительных затрат на подготовку тестовых кейсов. При этом данный метод менее применим на начальных этапах разработки, когда еще отсутствует полноразмерное окружение. [13] Многие современные инструменты мониторинга проектируются с учетом того, что их данные будут активно использоваться для проведения рефакторинга кода и улучшения архитектуры приложений. Такой подход позволяет разработчикам оперативно выявлять узкие места и оптимизировать производительность системы. [14]

1.2.2. Моделирование нагрузки на основе статистики

Этот подход базируется на анализе журналов, статистики выполнения запросов и накопленных данных эксплуатации системы. Используя такие инструменты, как *pg_stat_statements* и *auto_explain*, можно провести ретроспективный анализ и выделить закономерности, характерные для конкретных сценариев работы. Подобное статистическое моделирование позволяет оценить интенсивность работы подсистемы ввода-вывода без проведения дополнительных нагрузочных тестов. Однако такой анализ эффективен лишь при наличии достаточного объёма исторических данных, то есть в системах, которые уже работают в производственной среде. В ряде случаев могут возни-

кать сложности в интерпретации статистики, что требует аккуратной настройки инструментов сбора и анализа данных.

1.2.3. Теоретическое моделирование

Данный метод опирается на построение аналитических моделей, позволяющих оценивать время выполнения операций ввода-вывода с учётом характеристик таблиц, индексов и структур данных. Примеры таких моделей можно найти в работах Boncz, Stonebraker и других исследователей, где используются математические зависимости для прогнозирования производительности. Преимущество метода заключается в том, что оценка может быть проведена даже без непосредственного проведения нагрузочных тестов, что делает его удобным для ранних стадий проектирования. Вместе с тем требуются глубокие знания внутренней архитектуры СУБД и тщательная калибровка моделей с целью минимизации погрешностей, которые могут возникнуть при упрощении реальных рабочих сценариев. [15]

1.2.4. Коммерческие решения

На современном рынке представлено несколько готовых продуктов, специально разработанных для комплексного мониторинга и анализа производительности систем управления базами данных (СУБД). Примером таких систем являются *SolarWinds Database Performance Analyzer* и *Pivotal Greenplum Performance Monitor*. Данные решения предоставляют широкие возможности по сбору и визуализации данных о производительности. Кроме того, они поддерживают функцию оповещений в режиме реального времени, а также предоставляют рекомендации по оптимизации работы СУБД. [16] Характерной чертой таких коммерческих продуктов является их интеграция с более обширными системами управления IT-инфраструктурой, что значительно упрощает их использование в условиях изменяющейся нагрузки.

Однако следует отметить, что высокая стоимость внедрения этих решений и их основная ориентация на уже развернутые системы могут ограничивать их применение в новых проектах или при ограниченных бюджетах. В этом контексте особенное значение приобрета-

ет разработка инструментов, которые позволяют предсказывать нагрузку на диск без необходимости фактического запуска СУБД, что является критическим аспектом в стадии планирования систем с высокими требованиями к вводу-выводу.

Таблица 1 – Сравнение решений для мониторинга нагрузки на диск

Характеристика	**SolarWinds**	**Pivotal Greenplum**	**Redgate SQL Monitor**
Мониторинг нагрузки на диск	Да	Да	Да
Глубина анализа	Высокая	Средняя	Средняя
Визуализация использования	Да	Да	Да
Настройка оповещений	Да	Да	Ограниченная
Рекомендации по оптимизации	Да	Да	Нет

1.2.5. Использование методов машинного обучения

В последние годы всё более широкое распространение получают методы машинного обучения для мониторинга, анализа и предсказания нагрузки на дисковые устройства баз данных [17—19]. Машинное обучение используется для построения моделей, способных выявлять сложные нелинейные зависимости между характеристиками SQL-запросов, размером и локалностью данных, параметрами памяти и вычислительными ресурсами, с одной стороны, и степенью вовлечённости дисковой подсистемы PostgreSQL — с другой. [20] Такие модели, учитывающие широкий спектр метрик, позволяют не только анализировать текущую нагрузку, но и осуществлять её прогнозирование в различных сценариях эксплуатации системы.

Для построения моделей оценки нагрузки на диск используется как классическое машинное обучение (например, регрессионный анализ, метод опорных векторов, случайный лес), так и современные методы, основанные на глубоких искусственных нейронных сетях (LSTM, GRU, Transformer-модели). Применение нейросетевых архитектур особенно актуально для обработки временных рядов, отражающих эволюцию метрик производительности, таких как blks_read, blks_hit, xact_commit, tps получаемых из представлений статистики

PostgreSQL (`pg_stat_database`, `pg_stat_io`, и др.). В ряде работ предпринимаются попытки интеграции внешних факторов, таких как пики пользовательской активности, особенности шаблонов доступа, временные зависимости, что способствует повышению точности предсказаний[19].

В качестве целевой переменной модели чаще всего выступает один из количественных показателей нагрузки на диск: пропускная способность (IOPS), среднее время отклика, суммарное время ожидания операций ввода-вывода или другой интегральный показатель из статистики операционной системы (`iotop`, `vmstat`) либо внутренней статистики СУБД. Анализ данных включает как агрегацию метрик по временным окнам, так и более сложные методы feature engineering, направленные на выявление аномалий — например, резкое увеличение количества операций random read/write или неустойчивость latency.

Значительная часть современных исследований посвящена проблеме «предиктивного автоскейлинга» и интеллектуального управления ресурсами (Self-driving DBMS). [12; 21] С помощью методов машинного обучения реализуются системы, способные динамически выставлять параметры конфигурации файловой и буферной подсистемы PostgreSQL (например, `shared_buffers`, `effective_cache_size`, `work_mem`), предотвращая деградацию производительности за счёт упреждающей реакции на прогнозируемый рост нагрузки. Кроме того, подобные системы позволяют выявлять нетривиальные взаимосвязи между различными типами загрузки и рхитектурными особенностями аппаратной платформы (SSD, NVMe, HDD) [22].

Важным аспектом внедрения машинного обучения в мониторинг и прогнозирование нагрузки на диск является качество данных и сложность их интерпретации. Особое внимание уделяется предварительной очистке данных, обработке пропусков и выбросов, а также подбору релевантных признаков. Оценка точности разрабатываемых моделей обычно производится с применением метрик RMSE, MAE, Precision, Recall и других, а также посредством кросс-валидации на разнородных рабочих нагрузках (OLTP, OLAP).

Таким образом, машинное обучение открывает новые возмож-

ности для детального анализа и прогнозирования нагрузки на дисковую подсистему PostgreSQL. Указанные методы позволяют формировать интеллектуальные механизмы оптимизации, снижать риск возникновения «узких мест» и обеспечивать высокую производительность баз данных в условиях непредсказуемых изменений потока запросов. Интеграция подобных подходов с существующими инструментами мониторинга значительно расширяет потенциал управления ресурсами на корпоративных и облачных платформах [19; 22].

1.3 Промежуточные выводы

В результате проведенного анализа можно сделать следующие выводы:

- Эффективная оценка планируемой нагрузки на систему ввода-вывода является важнейшей задачей при проектировании информационных систем на базе PostgreSQL.
- Существующие методы в основном ориентированы на постфактум-анализ и не позволяют проводить предварительное прогнозирование нагрузки без фактической генерации запросов.
- Разработка специального средства оценки планируемой I/O-нагрузки на основе анализа структуры базы данных и характеристик запросов является актуальной и востребованной задачей.
- Для успешной реализации такого средства требуется глубокое понимание архитектуры PostgreSQL, процессов работы с данными и особенностей работы подсистемы ввода-вывода.

Итого, представленные подходы обладают своими сильными и слабыми сторонами, а выбор метода оценки I/O-нагрузки должен зависеть от конкретных условий эксплуатации и этапа жизненного цикла СУБД. Эти описания логически дополняют приведённую таблицу, предоставляя более глубокий анализ каждого метода и обосновывая критерии их выбора.

1.3.1. Сравнительная таблица существующих решений

Для наглядного сопоставления рассмотрим сводную таблицу, отражающую ключевые характеристики различных подходов:

Таблица 2 – Сравнение подходов к оценке нагрузки на подсистему ввода-вывода в PostgreSQL

Подход	**Типичные инструменты**	**Преимущества**	**Ограничения**
Профилирование реальной нагрузки	*pgbench*, *HammerDB*, кастомные скрипты	Наиболее приближено к реальной работе системы; позволяет наблюдать влияние конкретных операций	Требует развернутой среды; не подходит для ранних стадий проектирования
Моделирование на основе статистики	*pg_stat_statements*, *auto_explain*, *pgBadger*	Позволяет анализировать уже выполненные запросы и выявлять ”узкие места”	Не работает без накопленных данных эксплуатации
Теоретическое моделирование	Модели на основе параметров таблиц и индексов (например, Boncz, Stonebraker)	Может использоваться без запуска реальных запросов; полезно на этапе проектирования	Высокая сложность; возможны значительные отклонения от реального поведения
Операционный мониторинг	*iostat*, *vmstat*, *iotop*, *pg_stat_io* (PostgreSQL 16+)	Детализированная информация об I/O-активности на уровне ОС и СУБД	Сложно связать данные напрямую с бизнес-логикой или SQL-нагрузкой
Коммерческие решения	*SolarWinds DPA*, *Greenplum Monitor*, *Datadog*, *New Relic*	Готовая визуализация, алерты, прогнозы; часто поддерживают PostgreSQL из коробки	Дорогие; ориентированы на уже работающие системы

2 Особенности архитектуры PostgreSQL, влияющие на I/O-нагрузку

PostgreSQL как одна из самых популярных реляционных СУБД имеет архитектурные особенности, которые напрямую влияют на характер нагрузки на подсистему ввода-вывода:

- **Физическая организация хранения данных**: данные в PostgreSQL хранятся в виде таблиц и индексов в файловой системе. Каждая таблица или индекс представлен одним или несколькими файлами.
- **Механизм WAL (Write-Ahead Logging)**: все изменения сначала фиксируются в журнале предзаписи (WAL), что приводит к дополнительной нагрузке на запись.
- **Буферизация данных**: PostgreSQL использует собственный пул буферов для кеширования страниц данных в памяти, что снижает количество прямых обращений к диску.
- **Автоматические процессы обслуживания**: процессы *autovacuum* и *autoanalyze* выполняют фоновую работу по поддержанию базы данных в оптимальном состоянии, также внося вклад в I/O-нагрузку.
- **Методика выполнения запросов**: особенности планировщика запросов (*Query Planner*) и используемые ыим стратегии сканирования таблиц (sequential scan, index scan) влияют на количество операций чтения данных с диска.

Учитывая данные особенности, становится очевидным, что точная оценка I/O-нагрузки должна учитывать не только количество и характер запросов, но и внутренние механизмы работы PostgreSQL.

Ниже приведены развернутые описания каждого подхода, представленные выше в таблице. Текст дополняет информацию таблицы, раскрывая методологию и особенности каждого метода, не повторяя при этом данные, уже указанные в сравнительной характеристике.

2.1 Параметры конфигурации влияющие на нагрузку на диск

Таблица 3 – Параметры PostgreSQL влияющие на нагрузку на диск

Параметр	**Описание**
shared_buffers	Размер памяти для кэширования страниц базы данных. Меньший размер увеличивает количество обращений к диску.
work_mem	Память на операции сортировки и хеширования. При нехватке используется временный файл на диске.
effective_cache_size	Оценка доступного файлового кеша ОС для планировщика запросов.
wal_buffers	Буфер для временного хранения WAL-записей перед записью на диск.
checkpoint_segments	Количество WAL-сегментов между контрольными точками (устарело, заменено на max_wal_size).
max_wal_size	Максимальный размер WAL, при достижении которого инициируется чекпоинт.
bgwriter_lru_maxpages	Максимальное количество страниц, записываемых background writer-ом за один проход.
bgwriter_lru_multiplier	Множитель для вычисления количества страниц, записываемых при нехватке буферов.
synchronous_commit	Определяет, требуется ли подтверждение записи изменений на диск при коммите транзакции.
temp_buffers	Память, выделяемая на временные таблицы. При нехватке создаются временные файлы на диске.
log_temp_files	Порог логирования временных файлов. Помогает обнаружить чрезмерное использование диска временными файлами.

2.2 Структура хранения данных в PostgreSQL

Система управления базами данных PostgreSQL использует собственную файловую структуру для хранения данных, которая напрямую связана с логической моделью базы данных. Понимание физической организации данных является ключевым элементом для администрирования, оптимизации производительности и диагностики проблем, связанных с хранением и доступом к данным.

2.2.1. Таблицы и файлы: связь объектов БД с файлами

Каждая база данных в PostgreSQL представлена в файловой системе как каталог в директории `base/`, в котором хранятся файлы, соответствующие таблицам, индексам и другим объектам. Все объекты (таблицы, индексы и т.п.) имеют уникальный OID (object identifier), на основе которого создаётся имя файла. Например, таблица с OID 16423 будет храниться в файле `base/12345/16423`, где 12345 — OID базы данных.

Каждый такой файл представляет собой физическое представление таблицы. Индексы и TOAST-таблицы также представлены отдельными файлами. [23]

2.2.2. Каталожные таблицы и их физическое представление

Системные каталоги PostgreSQL (например, `pg_class`, `pg_attribute`) реализованы как обычные таблицы и хранятся в файловой системе наравне с пользовательскими объектами. Они находятся в системной базе данных `postgres` или `template1`, и также представлены отдельными файлами с числовыми именами (OID).

2.2.3. Сегментация файлов (segmenting) больших объектов

В PostgreSQL существует ограничение на максимальный размер одного файла хранения данных — по умолчанию 1 ГБ. Если размер таблицы или индекса превышает 1 ГБ, PostgreSQL автоматически делит его на сегменты. Эти сегменты именуются как `16423.1`, `16423.2`

и т.д., где `16423` — базовое имя файла, а суффиксы обозначают номер сегмента.

Это позволяет системе работать с очень большими таблицами, несмотря на ограничения ОС или файловой системы.

2.2.4. Механизм TOAST и хранение больших данных

TOAST (The Oversized-Attribute Storage Technique) — это механизм, предназначенный для хранения очень больших значений полей, таких как текст или бинарные объекты. При превышении определённого размера (по умолчанию 2 КБ), такие значения автоматически выносятся в отдельную TOAST-таблицу, которая также хранится как обычный файл, связанный с основной таблицей.

TOAST-таблицы используют собственные механизмы сжатия и разбиения данных на чанки (chunks), что позволяет эффективно управлять большими объектами без перегрузки основной таблицы.

2.2.5. Индексы и их файловая структура

Индексы в PostgreSQL также реализованы как отдельные файлы. Например, B-Tree индекс хранится в виде страниц фиксированного размера (по умолчанию 8 КБ), содержащих ключи и указатели. Как и таблицы, при превышении 1 ГБ, индекс делится на сегменты.

Каждому индексу соответствует запись в каталоге `pg_class`, а его физическое имя также строится по OID. Структура индекса зависит от его типа: B-Tree, Hash, GiST, GIN и т.д., но в любом случае он представлен в виде одного или нескольких сегментированных файлов.

2.3 Журналирование и Write-Ahead Logging (WAL) в PostgreSQL

Одним из основополагающих механизмов обеспечения надежности, согласованности данных и устойчивости к сбоям в системе управления базами данных PostgreSQL является организация журналирования на основе протокола Write-Ahead Logging (WAL). Этот механизм играет ключевую роль как при обычной работе с данными, так и в процессе восстановления после аварийных ситуаций.

2.3.1. Принцип Write-Ahead Logging

Write-Ahead Logging (WAL) — это протокол, согласно которому любые изменения, вносимые в основные файловые структуры базы данных (таблицы и индексы), сначала фиксируются в специальном журнальном файле (WAL-журнале) до того, как сами данные будут непосредственно записаны на диск. Это обеспечивает атомарность и долговечность операций, соответствуя принципам управления транзакциями (ACID). [24]

Ключевой особенностью подхода WAL является следующий момент: ни одно изменение, связанное с данными, не считается завершённым, пока информация об этом изменении не будет зафиксирована в журнале. Только после этого модификация может быть отражена в основной структуре базы данных. Такой подход позволяет в случае сбоя или неожиданного завершения работы системы восстановить базу данных до целостного и согласованного состояния посредством «проигрывания» (replay) зафиксированных в WAL операций. [25]

2.3.2. Роль WAL в оценке нагрузки и производительности

Журналирование посредством WAL напрямую влияет на нагрузку на подсистему ввода-вывода: интенсивность операций по записи в WAL часто определяет требования к дисковой подсистеме, особенно при высоких уровнях параллелизма транзакций. Анализ объёма, частоты и параметров WAL-записей позволяет прогнозировать и оценивать нагрузку на I/O, а также уточнять требования к оборудованию и настройкам сервера. [23]

Таким образом, механизм Write-Ahead Logging является фундаментом обеспечения надёжности PostgreSQL, а также критически важным фактором для оценки и прогнозирования интегральной нагрузки на систему ввода-вывода при проектировании и эксплуатации СУБД.

Таким образом, физическая организация PostgreSQL напрямую отражает логическую структуру базы данных. Глубокое понимание

связей между файлами и объектами БД позволяет администратору эффективно управлять хранением, планировать ресурсы и проводить тонкую настройку производительности.

3 Проектирование и разработка средства оценки планируемой нагрузки на систему ввода-вывода PostgreSQL

3.1 Постановка требований к разрабатываемому средству

Анализ существующих решений показывает, что ни один из подходов не ориентирован напрямую на предварительную оценку I/O-нагрузки без реального выполнения запросов. Поэтому к разрабатываемому средству предъявляются следующие требования:

- **Прогнозирование без запуска нагрузки**: инструмент должен уметь оценивать предполагаемую нагрузку по метаданным базы данных и характеристикам планируемых запросов.
- **Поддержка типовых операций PostgreSQL**: необходимо учитывать особенности выполнения основных операций: вставка и выборка.
- **Оценка объема операций чтения и записи**: средство должно отдельно оценивать предполагаемые объемы чтения и записи данных.
- **Модульность и расширяемость**: архитектура решения должна позволять легко адаптировать его к новым версиям PostgreSQL и различным типам приложений.
- **Простота использования**: инструмент должен быть доступен для использования специалистами без глубокого знания внутренней архитектуры PostgreSQL.

3.2 Разработка программного средства

3.2.1. Выбор инструментов и технологий

Выбор инструментов и технологий, использованных при разработке программного средства, был обусловлен как специфическими требованиями предметной области (прогнозирование нагрузки на СУБД PostgreSQL), так и общими соображениями масштабируемости, воспроизводимости и автоматизации. В данном подразделе приведён

обоснованный выбор программных средств, языков программирования, форматов представления данных и средств автоматизации.

Язык программирования Python В качестве основного языка реализации был выбран Python. Основные причины выбора следующие:

- **Высокий уровень абстракции**. Python предоставляет удобные средства для работы со строками, файлами, регулярными выражениями и структурами данных, что особенно актуально при анализе SQL-описаний (DDL) и конфигурационных файлов.
- **Богатая экосистема**. Используемые стандартные библиотеки (`yaml`, `re`, `math`, `argparse`) полностью покрывают потребности проекта без необходимости в дополнительных зависимостях.
- **Удобство для прототипирования**. Язык позволяет быстро разрабатывать и отлаживать модули, что особенно важно на этапе научных экспериментов и итеративной разработки.

Формат конфигурации: YAML Для задания входных параметров алгоритма был выбран формат YAML по следующим причинам:

- **Читаемость**. YAML близок к естественному синтаксису и легко воспринимается человеком, что снижает порог входа для конечных пользователей и разработчиков.
- **Иерархичность**. Формат удобно отражает вложенные структуры, такие как описание нескольких таблиц с DDL, параметры нагрузки и конфигурацию окружения.
- **Совместимость с Python**. Модуль `PyYAML` обеспечивает простую загрузку данных YAML в структуры Python (словарь, список и т.д.).

Shell-скрипты и автоматизация Для автоматизации запуска программного средства и обработки множественных конфигурационных файлов использовались скрипты командной оболочки Bash. Их функции включают:

- **Циклический запуск анализа** по множеству входных YAML-файлов.
- **Агрегация и вывод результатов** в удобном табличном формате.
- **Интеграция с системой CI/CD**, в частности, в рамках GitHub Actions.

Использование Bash позволяет не привязывать обработку к конкретной ОС, поскольку все компоненты являются кроссплатформенными и не требуют сторонних зависимостей.

GitHub Actions Система GitHub Actions используется для автоматизации тестирования и анализа кода в момент каждого коммита или пулл-запроса. Преимущества выбора данной технологии:

- **Автоматический запуск проверок** при каждом изменении кода.
- **Гарантия воспроизводимости** за счёт запуска в изолированном контейнерном окружении.
- **Интеграция с репозиторием** без необходимости использования внешних CI-серверов.

В рамках проекта реализован отдельный workflow-файл, который инициирует запуск Bash-скрипта, передаёт необходимые параметры и сохраняет результаты анализа.

Пользовательский web-интерфейс: React + JavaScript Для повышения удобства взаимодействия с программным средством и расширения целевой аудитории был разработан современный web-интерфейс на основе библиотеки React. Данный компонент реализует пользовательскую форму для задания входных данных: структуры таблиц, конфигурации PostgreSQL, параметров нагрузки и запросов.

Основные причины выбора React:

- **Динамичность интерфейса**. React обеспечивает реактивное обновление состояния и рендеринг формы в зависимости от пользовательских действий, что критически важно в задачах интерактивного конфигурирования параметров.

- **Поддержка сложных форм**. Библиотека позволяет удобно реализовывать динамические добавления и редактирование таблиц, параметров, выводить подсказки и валидировать вводимые значения на лету.

- **Развитая экосистема**. Использование сторонних библиотек (например, Papaparse для обработки CSV с подсказками по конфигурации) существенно упрощает реализацию и расширение функционала.

- **Кроссплатформенность и доступность**. React-приложение легко развёртывается как отдельный SPA, интегрируется с серверной частью через REST API или может разворачиваться как statically-served сайт в любом окружении.

- **Масштабируемость**. Архитектура React позволяет в будущем легко интегрировать новые функции: графики (например, через Plotly), автоматическую проверку корректности вводимых DDL или расширенные формы для разных сценариев нагрузки.

Базовый функционал разработанного интерфейса включает:

- Формирование и редактирование списка таблиц с интуитивным вводом DDL и числа строк;

- Автоматизированный подбор и изменение параметров конфигурации PostgreSQL с поясняющими подсказками, загружаемыми из справочных данных;

- Ввод тестового SQL-запроса и параметра нагрузки (RPS);

- Поддержку вложенных, типизированных и связанных параметров (разделение числовых значений и единиц измерения, on/off-переключатели, списки и т.д.).

Реализация приложения на React позволила обеспечить современный UX и повысить скорость конфигурирования, а также снизить требования к технической подготовке конечного пользователя, по сравнению с ручным редактированием YAML-файлов.

Вывод Таким образом, совокупность выбранных технологий обеспечивает гибкость, воспроизводимость, автоматизируемость и пользовательскую доступность разработанного программного средства. При этом архитектура построена таким образом, что допускает расширение функциональности и адаптацию под иные СУБД или сценарии нагрузки в будущем.

3.2.2. Описание алгоритма прогнозирования

Разработанный алгоритм предназначен для прогнозирования объёма дискового пространства, необходимого для хранения пользовательских данных в системе управления базами данных PostgreSQL, а также для предварительной оценки предполагаемой нагрузки (в терминах RPS — запросов в секунду) на отдельные компоненты файловой подсистемы. Алгоритм реализован на языке программирования Python и ориентирован на статический анализ конфигурационных данных и DDL-описаний таблиц.

Основными этапами работы алгоритма являются:

1. **Извлечение структурной информации из DDL-описания.** С использованием регулярных выражений производится синтаксический разбор SQL-команды создания таблицы (`CREATE TABLE`). Алгоритм выделяет список столбцов с соответствующими типами данных, а также определяет тип индекса, если он указан явно (например, `BTREE`, `GIN`, `HASH` и др.). Предусмотрена обработка ключевых слов, указывающих на составные ключи и особенности индексирования.

2. **Оценка объёма хранения одной строки таблицы.** Для каждого столбца производится оценка объёма занимаемой памяти

в байтах. В рамках текущей реализации реализована эвристическая схема оценки:

- целочисленные типы (`INT`, `BIGINT`) оцениваются в 4 байта;
- типы `NUMERIC` — в 16 байт;
- типы временных меток (`TIMESTAMP`) — в 8 байт;
- символьные типы (`CHAR(n)`) — в n байт;
- переменные по длине типы, такие как `TEXT` и `JSON`, принимаются равными 2000 и 3000 байт соответственно.

Кроме того, добавляется фиксированный системный оверхед в 24 байта на каждую строку.

3. **Учет механизма TOAST.** В случае, если совокупный размер строки превышает установленный порог (в PostgreSQL это обычно 2000 байт), предполагается использование механизма TOAST (The Oversized-Attribute Storage Technique). В таком случае создаётся отдельная сущность хранения, и её объём также включается в итоговую оценку. Размер строки в основной таблице после вынесения больших атрибутов оценивается условно в 50 байт.

4. **Расчет плотности хранения и общего количества страниц.** Учитывается размер страницы PostgreSQL (`PAGE_SIZE` = 8192 байт) и коэффициент заполнения страницы (`FILL_FACTOR` = 0,85). На основе рассчитанного размера строки определяется количество строк на одной странице и далее — общее число страниц, необходимое для хранения всех строк таблицы.

5. **Формирование структуры хранения.** Для каждой таблицы формируется совокупность логических файлов, соответствующих различным компонентам системы хранения PostgreSQL:

   - основной heap-файл таблицы;
   - TOAST-таблица (при необходимости);
   - индексный файл (в зависимости от типа индекса);

- карты свободного пространства (FSM — Free Space Map);
- карты видимости (VM — Visibility Map).

Для каждого из указанных компонентов вычисляется примерный размер в байтах и фиксируется предполагаемый тип доступа (например, “Heap access (join read)”, “Index (btree)”, “TOAST read/write” и др.).

6. **Оценка нагрузки (RPS).** В конфигурационном файле YAML указывается SQL-запрос, который будет исполняться в рамках моделируемой нагрузки. Алгоритм выделяет из запроса имена задействованных таблиц (анализируя конструкции `FROM` и `JOIN`) и помечает соответствующие таблицы как “активные”. Только активным таблицам присваивается значение RPS, указанное в конфигурации. Для неактивных таблиц нагрузка считается нулевой.

7. **Вывод и визуализация результатов.** На выходе алгоритм формирует таблицу, содержащую информацию по каждому логическому файлу: имя, размер в килобайтах, значение RPS и характер доступа. Эта информация может быть выведена как в консольном интерфейсе, так и, при интеграции с веб-приложением, в виде интерактивной HTML-таблицы.

Таким образом, предложенный алгоритм позволяет без запуска СУБД и исполнения реальных SQL-запросов получить оценку распределения нагрузки на файловую подсистему PostgreSQL. Это делает его полезным инструментом как для предварительного анализа схемы данных, так и для целей тестирования, анализа производительности и оптимизации структуры хранения.

3.2.3. Формат входных данных основного алгоритма

Входные данные для алгоритма прогнозирования нагрузки на диск представляются в виде конфигурационного файла в формате YAML.

Данный файл отражает как логическую структуру обрабатываемых данных, так и параметры выполнения запроса, характеристики окружения и конфигурационные настройки PostgreSQL, влияющие на поведение подсистемы хранения. Формат входного файла структурирован и предназначен как для автоматической обработки, так и для удобства ручного редактирования.

Файл включает четыре логических блока: `tables`, `load_generator`, `postgresql_config`, `environment`.

Блок `tables` Раздел `tables` представляет собой список таблиц, каждая из которых описывается следующими параметрами:

- `name` — строка, задающая уникальное имя таблицы.
- `ddl` — SQL-выражение `CREATE TABLE`, задающее схему таблицы (с типами столбцов, индексами, ключами).
- `row_count` — предполагаемое количество строк в таблице, необходимое для оценки объёма хранимых данных и числа используемых страниц.

Алгоритм парсит DDL-описание таблиц для определения размера строк, необходимости TOAST-хранилища, типа индексной структуры и особенностей хранения массивов и JSONB-типов.

Блок `load_generator` Данный блок описывает модель генерируемой нагрузки на базу данных, имитируя поведение прикладного уровня:

- `query` — SQL-запрос, на основе которого осуществляется прогнозирование интенсивности обращений к таблицам. Из запроса извлекаются участвующие в соединениях таблицы и предполагаемая направленность доступа (например, только чтение).
- `rps` (requests per second) — оценка числа запросов в секунду, поступающих к указанному SQL-запросу. Данный параметр масштабирует итоговую нагрузку, распределяя её между таблицами и индексами, участвующими в запросе.

Блок `postgresql_config` Конфигурация PostgreSQL включает все основные параметры, оказывающие влияние на поведение подсистемы хранения данных. Среди них:

- **Буферизация и кэширование**:
  - `shared_buffers` — объём оперативной памяти, зарезервированный под буферы PostgreSQL.
  - `effective_cache_size` — оценка размера файлового кэша, доступного PostgreSQL со стороны ОС.
  - `work_mem`, `maintenance_work_mem` — лимиты памяти на операции сортировки и создания индексов.
- **Конфигурация WAL и контрольных точек (checkpoints)**:
  - `wal_buffers`, `max_wal_size`, `min_wal_size`, `wal_compression` — параметры объёма и сжатия журналов WAL.
  - `checkpoint_timeout`, `checkpoint_completion_target` — параметры частоты и длительности контрольных точек.
  - `fsync`, `synchronous_commit` — параметры, определяющие, требуется ли жёсткая синхронизация данных на диск при коммите.
- **Автовакуум и фоновая активность**:
  - `autovacuum`, `autovacuum_max_workers`, `autovacuum_naptime` — параметры частоты и параллелизма фоновой очистки таблиц.
  - `autovacuum_vacuum_cost_limit`, `autovacuum_vacuum_cost_delay` — параметры скорости фонового VACUUM.
- **Оценки стоимости операций**:
  - `random_page_cost`, `seq_page_cost` — логические параметры, отражающие относительную стоимость случайного и последовательного чтения страниц. Используются в планировщике запросов и влияют на стратегию доступа к данным.

- **Прочие параметры**:
  - `wal_level` — уровень детализации WAL, определяющий объём логирования и, соответственно, нагрузку на диск.
  - `wal_writer_delay` — задержка между фоновыми сбросами WAL на диск.
  - `temp_buffers` — объём памяти, выделяемый на временные таблицы, участвующие в запросах.

Блок `environment` Описывает характеристики вычислительного окружения, на котором предполагается выполнение анализа:

- `cpu_cores` — количество доступных процессорных ядер.
- `ram_gb` — общий объём оперативной памяти в гигабайтах.
- `storage_type` — тип дисковой подсистемы (например, `ssd`, `hdd`, `nvme`).
- `os_cache_enabled` — булев флаг, указывающий, доступен ли файловый кэш со стороны ОС.

Вывод Предложенный формат конфигурационного файла обеспечивает достаточную полноту и выразительность для оценки поведения системы хранения PostgreSQL при различных сценариях нагрузки. YAML-представление обеспечивает читаемость, расширяемость и лёгкую интеграцию с автоматическими средствами анализа и визуализации.

Листинг 3.1 — Содержимое файла `example1.yaml`

```
load_prediction_config:
  tables:
    - name: users
      ddl: |
        CREATE TABLE users (
          id SERIAL PRIMARY KEY,
```

```
        email TEXT,
        interests TEXT[],
        profile JSONB
      );
    row_count: 500000

  - name: orders
    ddl: |
      CREATE TABLE orders (
        id SERIAL PRIMARY KEY,
        user_id INT,
        amount NUMERIC,
        status TEXT
      );
    row_count: 1200000

load_generator:
  query: |
    SELECT u.email, o.amount
    FROM users u
    JOIN orders o ON u.id = o.user_id
    WHERE u.interests @> ARRAY['tech'] AND o.status
      = 'paid';
  rps: 5

postgresql_config:
  ...

environment:
  cpu_cores: 2
  ram_gb: 2
  storage_type: "ssd"
```

3.2.4. Формат выходных данных алгоритма предсказания нагрузки на систему ввода-вывода

Выходные данные алгоритма предсказания нагрузки на подсистему ввода-вывода представлены в формате YAML, отражающем структурированное описание файловых компонентов системы управления базами данных PostgreSQL, участвующих в операциях хранения и извлечения данных. Формат включает параметры физического хранения, объёмы, характер обращений и модель оценки производительности в условиях различной эффективности кэширования.

Каждый логический файл базы данных описывается в отдельной структуре с обязательным указанием следующих атрибутов:

- `name` — абсолютный путь к файлу в файловой системе PostgreSQL;
- `type` — тип файла: основной (heap), вспомогательный (TOAST), индексный (btree_index), карта свободного пространства (fsm);
- `estimated_size_mb` — предполагаемый объём файла в мегабайтах;
- `access_pattern` — описание характера обращений к файлу, включая совокупное количество операций (`total_operations`) и сценарии обращения при различных коэффициентах попадания в кэш;
- `io_scenarios` — набор параметров, описывающих производительность при заданной доле кэшированных операций (`cache_hit_ratio`). Для каждого сценария указываются:
  - `read_ops` — количество операций чтения, направленных непосредственно к устройствам хранения;
  - `write_ops` — количество операций записи;
  - `read_throughput_mb_s` — оценка пропускной способности по чтению (в мегабайтах в секунду);
  - `write_throughput_mb_s` — оценка пропускной способности по записи.

Алгоритм учитывает четыре основные модели кэширования:

1. 0% попаданий в кэш (все обращения направлены к физическому диску);
2. 25% попаданий в кэш;
3. 50% попаданий в кэш;
4. диапазон от 50% до 80% попаданий в кэш, имитирующий реальное поведение при высокоэффективных буферных пулах.

Дополнительно структура YAML содержит секцию `metadata`, в которой указываются единицы измерения параметров и поясняющие примечания.

Пример соответствующего YAML-файла приведён ниже:

Листинг 3.2 — Содержимое файла `output-example.yaml`

```yaml
postgres_disk_io_prediction:
  database: my_database
  tablespace: pg_default
  files:
    - name: base/16384/12345
      type: heap
      estimated_size_mb: 2048
      access_pattern:
        total_operations: 500000
        io_scenarios:
          - cache_hit_ratio: 0.0
            read_ops: 300000
            write_ops: 200000
            read_throughput_mb_s: 80
            write_throughput_mb_s: 40
          ...
metadata:
  units:
    size: MB
```

```
20      throughput: MB/s
21      operations: iops (read/write)
22    notes:
23      - "cache_hit_ratio refers to percentage of data
          read from memory (not disk)"
24      - "Estimated throughput is approximate and
          hardware dependent"
```

Данный формат обеспечивает пригодность для последующего анализа, визуализации и генерации нагрузочного тестирования в системах моделирования ввода-вывода или автоматизированных средствах администрирования СУБД PostgreSQL.

3.3 Реализация web-интерфейса

3.3.1. Основные требования к web-интерфейсу

Одной из главных задач при разработке средства оценки планируемой нагрузки на систему ввода-вывода СУБД PostgreSQL является создание интуитивно понятного и функционального web-интерфейса, обеспечивающего пользователю удобный ввод параметров моделирования и получение рекомендаций по настройке среды. Интерфейс должен позволять пользователю задавать параметры конфигурации СУБД PostgreSQL, а также характеристики используемых таблиц и SQL-запросов, что необходимо для построения корректной модели нагрузки.

3.3.2. Выбор инструментов реализации

В качестве основы для реализации web-интерфейса был выбран фреймворк React, известный своей модульностью, высоким уровнем абстракции и поддержкой современных парадигм разработки SPA (Single-page Application). Для удобной работы с CSV-файлами использовалась библиотека `papaparse`, посредством которой реализована подгрузка текстовых подсказок (hints) по настройкам из внешнего файла с рекомендациями. Также применялись инструменты для унификации и стилизации пользовательского интерфейса (CSS, Tailwind).

3.3.3. Архитектура и компоненты интерфейса

Разработанный web-интерфейс логически состоит из следующих основных модулей:

- форма для ввода перечня таблиц, хранящихся в СУБД, их определяющих конструкций (DDL) и приблизительных размеров (количества строк);
- панель настройки параметров конфигурации PostgreSQL с разделением полей по единицам измерения (байты, время, бинарные параметры);
- секция для задания SQL-запроса и интенсивности работы (RPS — запросы в секунду).

Структура интерфейса разбита на секции для повышения читаемости и модульности. Взаимодействие между компонентами осуществляется посредством *состояний* (React Hooks), что обеспечивает мгновенное обновление данных при изменениях, совершённых пользователем.

На рисунке 1 изображён внешний вид основной страницы web-интерфейса.

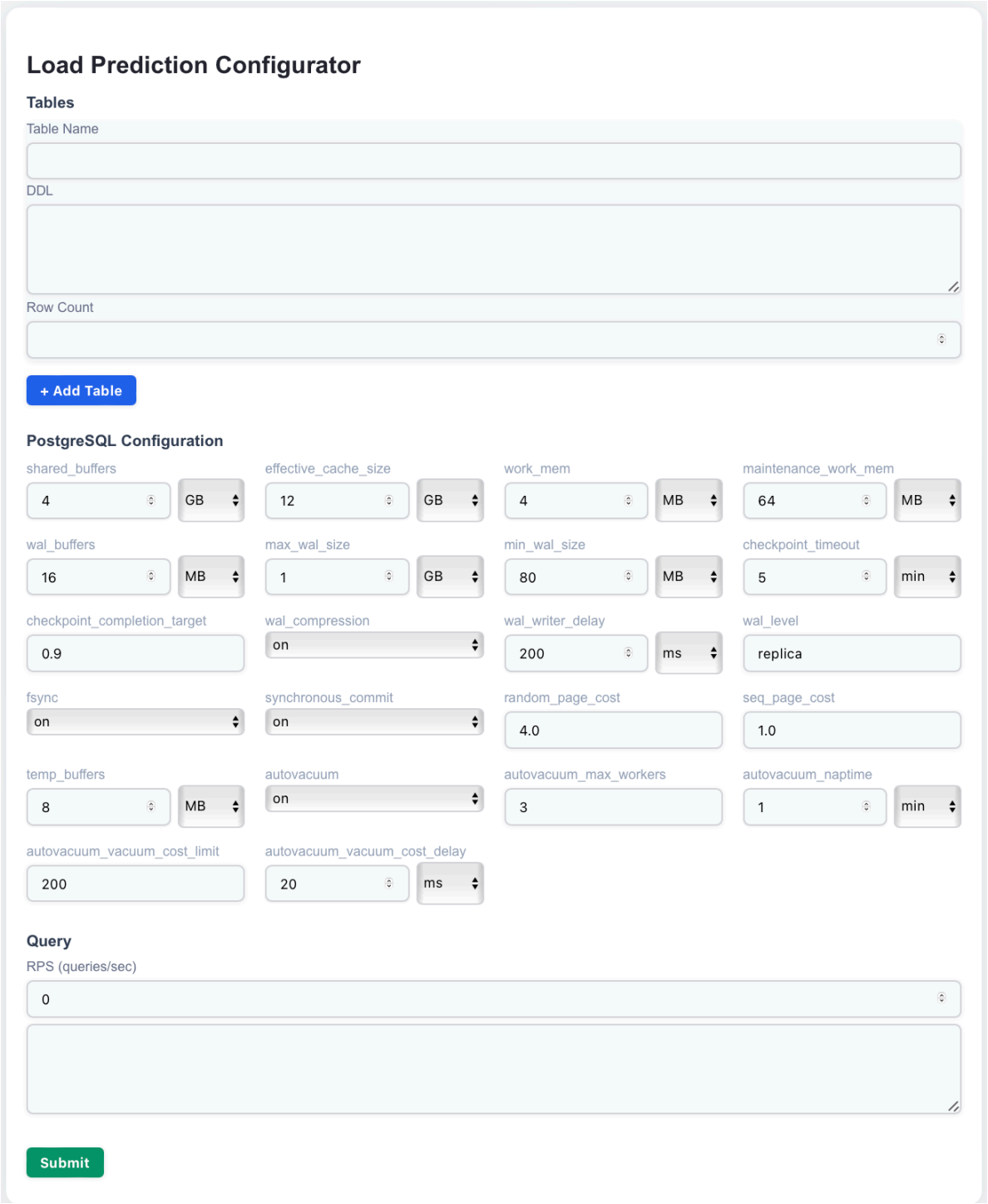

Рис. 1 – Внешний вид реализованного web-интерфейса средства оценки нагрузки.

3.3.4. Ввод информации о таблицах

Пользователь может добавить неограниченное количество схем таблиц, каждая из которых описывается тройкой параметров:

- название таблицы (*Table Name*);
- определяющая конструкция DDL (*DDL*), указывается в формате

SQL;

- предполагаемое количество строк (*Row Count*), что критически важно для моделирования нагрузки.

Для добавления очередной таблицы реализована кнопка *“+ Add Table”*. Изменения применяются немедленно, значения инкапсулируются в соответствующем состоянии.

3.3.5. Панель настройки параметров PostgreSQL

Особое внимание уделяется корректной обработке различных типов параметров конфигурации СУБД:

- параметры включения/выключения (*on*/*off*) — выводятся в виде выпадающих списков;
- временные и объёмные (байтовые) параметры разбиты на числовую часть и единицу измерения, что минимизирует ошибки пользователя;
- остальные параметры визуализируются в виде текстовых полей.

Каждое поле сопровождается интерактивной подсказкой, расположенной под ним, если для данного ключа имеется поясняющий текст в подгруженном `postgres_config_hints.csv`. Это способствует правильному выбору пользователем подходящих значений.

3.3.6. Ввод SQL-запроса и интенсивности работы

В специальной секции пользователь вводит синтаксис моделируемого SQL-запроса (или его шаблон), а также задаёт интенсивность нагрузки (RPS, запросы в секунду). Эти данные необходимы для последующего прогноза нагрузки на систему ввода-вывода.

3.3.7. Валидация и обработка введённых данных

Реализация контролирует тип полей (текст, число, выпадающий список), диапазон вводимых значений, корректность комбинации чис-

ла и единицы измерения. Вся логика обработки ввода инкапсулирована в компоненте приложения, состояние синхронизируется с действиями пользователя.

3.3.8. Взаимодействие с серверной частью

После нажатия кнопки *Submit* собранные параметры отправляются на сервер для последующей обработки. Серверная логика, реализованная отдельно, использует полученные данные для анализа и моделирования I/O-нагрузки СУБД PostgreSQL.

3.3.9. Резюме по реализации интерфейса

Итоговая реализация web-интерфейса позволяет гибко задать все существенные параметры, необходимые для моделирования нагрузки и последующего анализа. Структура интерфейса обеспечивает как простоту начального освоения, так и расширяемость. Выбранные технологические решения и разделение логики на отдельные компоненты обеспечивают независимость модулей, устойчивость к ошибкам и возможность масштабирования при развитии проекта.

3.4 Автоматизация запуска с помощью Bash и интеграция с GitHub Actions

Автоматизация рутинных задач является важной частью современного программирования, позволяя разработчикам сосредоточиться на решении более сложных и творческих задач. В данном разделе рассмотрим процесс автоматизации запуска скрипта с использованием Bash и его интеграцию в процесс CI/CD посредством GitHub Actions.

Структура проекта

Проект структурирован таким образом, чтобы обеспечить простую навигацию и легкость понимания. Корневая директория содержит основной скрипт запуска, `run-examples.py`, который представляет собой Bash-скрипт для управления виртуальным окружением Python и выполнения анализатора. В корневой директории также находится

файл зависимостей `requirements.txt`, а примеры, предназначенные для анализа, располагаются в подкаталогах, соответствующих шаблону `examples/example*/ *.yaml`.

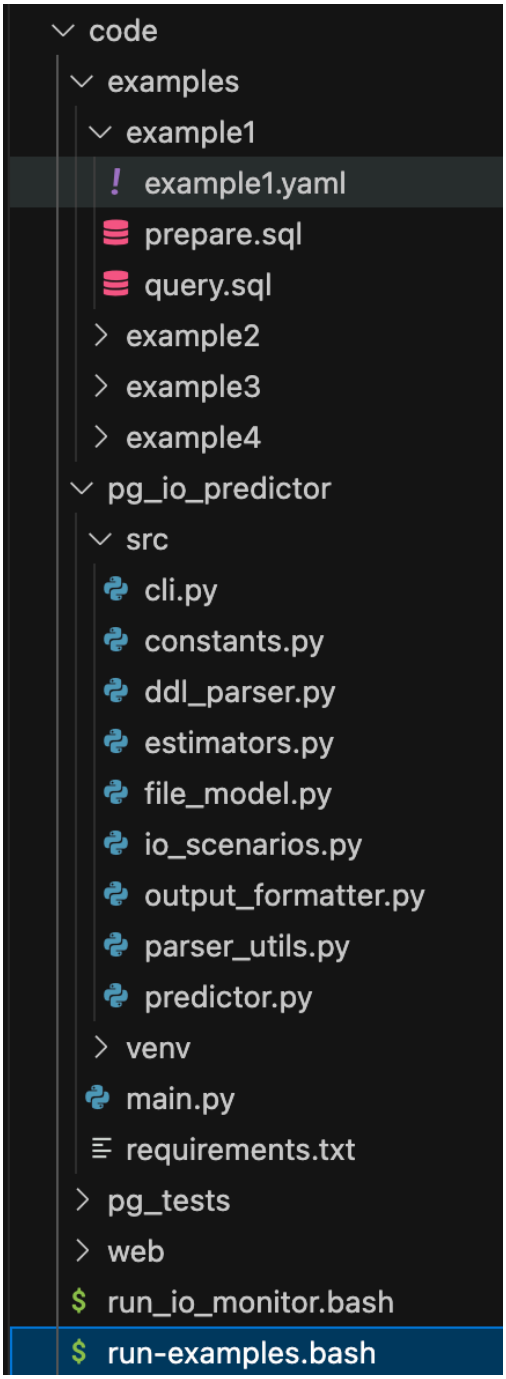

Рис. 2 – Структура файлов в проекте.

Описание `run-examples.py`

Скрипт `run-examples.py` выполняет несколько ключевых задач:

Создание виртуального окружения: Если директория `venv` отсутствует, создается новое виртуальное окружение. Активация окружения: Виртуальное окружение активируется посредством команды

source. Обновление pip: Перед установкой зависимостей обновляется пакетный менеджер pip. Установка зависимостей: Зависимости устанавливаются из файла requirements.txt, если он доступен. Анализ YAML-файлов: Скрипт последовательно проходит по каждому YAML-файлу в директории примеров и запускает анализатор analyze_pos Деактивация окружения: Завершает работу, деактивируя виртуальное окружение. Каждый шаг содержит проверку на наличие ошибок, что позволяет скрипту надёжно обрабатывать исключительные ситуации, делая его более стабильным и надежным.

Интеграция с GitHub Actions

GitHub Actions предоставляет мощные инструменты для автоматизации рабочих процессов. Интеграция Bash-скрипта в процесс CI/CD с помощью GitHub Actions позволяет запускать анализатор автоматически при каждом коммите. Ниже приведен пример конфигурации рабочего процесса:

Листинг 3.3 — Содержимое файла actions.yaml

```
name: run examples

on: [push]

jobs:
  code-tests:
    if: ${{ github.event_name == 'push' || github.
      event_name == 'pull_request' }}
    runs-on: ubuntu-latest
    name: Run code tests
    #     ,              -   code/
    steps:
      - name: Checkout repository
        uses: actions/checkout@v3

      - name: Check for code/ changes
        id: changes
```

```
17          uses: dorny/paths-filter@v2
18          with:
19            filters: |
20              code:
21                - 'code/**'
22
23        - name: Set up Python
24          if: steps.changes.outputs.code == 'true'
25          uses: actions/setup-python@v4
26          with:
27            python-version: '3.10'
28
29        - name: Install dependencies
30          if: steps.changes.outputs.code == 'true'
31          run: |
32            pip3 install -r code/requirements.txt
33
34        - name:  Run analyzer on all examples
35          run: |
36            cd code
37            bash run-examples.bash
```

3.5 Резюме по реализации программного средства

В данной работе было реализовано программное средство для оценки объёмов и интенсивности обращения к дисковым структурам в системе управления базами данных PostgreSQL на основе анализа схемы данных, предполагаемой нагрузки и параметров окружения.

Разработка выполнена с применением интерпретируемого языка Python, что обеспечило гибкость, расширяемость и удобство интеграции с другими компонентами инфраструктуры. Основной модуль программы реализует алгоритм прогнозирования, который производит анализ описания таблиц в формате DDL, рассчитывает размер строк, распределение данных по страницам и необходимость использования механизмов TOAST. Также учитываются вторичные струк-

туры хранения, включая индексы различных типов (btree, GIN, GiST и др.), карты свободного и видимого пространства (FSM и VM), а также влияние параметров буферизации.

Входные данные подаются в формате YAML, что позволяет обеспечить наглядную и декларативную спецификацию характеристик базы данных и параметров нагрузки. Алгоритм автоматически интерпретирует SQL-запрос, извлекая перечень таблиц, участвующих в операциях соединения, что критически важно для корректного распределения запросов (RPS) между различными структурами хранения.

В рамках реализации обеспечена автоматизация процессов запуска: реализованы вспомогательные Bash-скрипты, позволяющие последовательно обрабатывать набор тестовых конфигураций. Интеграция с системой непрерывной интеграции GitHub Actions обеспечивает повторяемость и прозрачность выполнения тестов при каждом изменении кода.

Дополнительно был реализован прототип web-интерфейса, предоставляющий пользователю возможность загружать конфигурационные файлы, запускать анализ и визуализировать результаты прогнозирования в удобной форме. Это делает разработанное средство пригодным как для целей экспериментальной оценки, так и для использования в образовательных и консалтинговых задачах по проектированию архитектуры СУБД.

Таким образом, полученное программное решение представляет собой законченный и воспроизводимый инструмент, позволяющий на основе декларативного описания схемы и нагрузки предварительно оценивать нагрузку на подсистему хранения данных PostgreSQL без необходимости реального развёртывания полноценных нагрузочных тестов.

4 Тестирование и апробация разработанного решения

4.1 Методика апробации и валидирования разрабатываемого средства

В целях всесторонней оценки корректности и практической применимости разработанного программного средства для оценки планируемой нагрузки на систему ввода-вывода СУБД PostgreSQL была реализована процедура тестирования, состоящая из нескольких этапов, включающая автоматизированный сбор статистики ввода-вывода на уровне самой СУБД, операционной системы, а также сопоставление полученных результатов с прогнозными значениями, рассчитываемыми предлагаемой программой. Для автоматизации получения метрик на выделенном сервере был применён специально разработанный скрипт (листинг приведён в приложении), обеспечивающий последовательное выполнение следующих действий:

- выбор тестового сценария из подготовленных примеров;
- автоматизированная подготовка данных в тестовой базе (выполнение DDL-операций и наполнение содержимым);
- запуск серии запросов к PostgreSQL с параметризированной интенсивностью (RPS – requests per second) и продолжительностью нагрузки;
- сбор статистики ввода-вывода посредством штатных средств СУБД PostgreSQL (динамические представления `pg_stat_io`, `pg_statio_user_tables`), а также средствами операционной системы (`/proc/PID/io`, `iostat`);
- накопление и анализ дифференциальных показателей между состояниями до и после нагрузки.

Полученные в процессе автоматических тестовых прогонах метрики сравнивались с результатами, выдаваемыми предсказывающим алгоритмом, реализованным в рамках дипломного проекта. Совокупность собранных данных позволила осуществить сопоставление точности расчетных и фактических величин нагрузки на I/O-подсистему.

Следует отметить, что в данном разделе изложены только общие принципы и последовательность испытаний.

Детализированное изложение указанных аспектов позволит объективно оценить эффективность и практическую значимость разрабатываемого программного средства в условиях разнообразных сценариев эксплуатации СУБД PostgreSQL.

4.2 Описание тестовых примеров и сценариев

Таблица 4 – Сравнительная характеристика сценариев тестирования PostgreSQL

№	Табличная структура (DDL и объём)	RPS	Индексы	TOAST	Тип нагрузки
1	users (500 000), orders (1 200 000)	5	PK, FK	да	SELECT
2	sensor_data (100 000 000)	3	PK	нет	SELECT
3	accounts (200 000)	1	PK, UNIQUE	нет	SELECT
4	logs (растет с 0)	2	PK	да	INSERT

Пояснения:

- В графе “TOAST” указано, требуется ли для сценария механизм хранения крупногабаритных данных (TOAST), см. [26].
- “PK” — первичный ключ; “FK” — внешний ключ; “UNIQUE” — уникальный индекс.
- Объёмы приведены в количестве строк таблицы согласно сценариям.
- Если поле типа JSONB или TEXT может содержать данные, превышающие одну страницу (8 КБ), PostgreSQL применяет TOAST [23].

4.3 Тестовая среда и ресурсы

Тестирование проводилось на выделенном сервере, предоставленном одним из популярных провайдеров облачных решений. Система управления базами данных (СУБД) была развернута на опера-

ционной системе Ubuntu без дополнительных оптимизаций конфигурации, что позволило оценить её производительность в условиях стандартной настройки. Такой подход обеспечил объективность результатов тестирования, исключив влияние сторонних модификаций и специализированных тюнинговых параметров.

Листинг 4.1 — Конфигурация тестового стенда `stand.yaml`

```yaml
environment:
  hardware:
    cpu:
      model:
        Model name: "Intel Xeon Processor (Icelake)"
        CPU family: 6
        Model: 106
      core_per_socket: 1
      threads_per_core: 2
      cache:
        l1: "64 KiB (2 instances)"
        l2: "4 MiB (1 instance)"
        l3: "16 MiB (1 instance)"

    memory:
      total: "2 GB"
      swap: "0 B"

    disks:
      - model: "Samsung SSD 870 EVO 500GB"
        type: "SSD"
        size: "80 GB"
        filesystem: "ext4"
        scheduler: "deadline"
        iops: 1500

  os:
```

```
    name: "Ubuntu"
    version: "24.04 noble"
    kernel: "Linux 6.8.0-59-generic"
    architecture: "x86_64"
```

В приведённом ниже фрагменте конфигурационного файла PostgreSQL представлены только те параметры, которые были явно заданы в стандартной поставке системы управления базами данных (СУБД). Остальные настройки соответствуют значениям по умолчанию, предусмотренным базовой конфигурацией. В ходе тестирования параметры СУБД не изменялись, за исключением случаев, когда их модификация была явно указана в условиях эксперимента.

Листинг 4.2 — Часть конфигурации СУБД

```
environment:
  hardware:
    cpu:
      model:
        Model name: "Intel Xeon Processor (Icelake)"
        CPU family: 6
        Model: 106
      core_per_socket: 1
      threads_per_core: 2
      cache:
        l1: "64 KiB (2 instances)"
        l2: "4 MiB (1 instance)"
        l3: "16 MiB (1 instance)"

    memory:
      total: "2 GB"
      swap: "0 B"

    disks:
      - model: "Samsung SSD 870 EVO 500GB"
```

```
21          type: "SSD"
22          size: "80 GB"
23          filesystem: "ext4"
24          scheduler: "deadline"
25          iops: 1500
26
27    os:
28      name: "Ubuntu"
29      version: "24.04 noble"
30      kernel: "Linux 6.8.0-59-generic"
31      architecture: "x86_64"
```

Более полный файл конфигурации PostgreSQL представлен в (см. А), если в нем нет искомого параметра, то параметр имеет стандартное значение для данной версии СУБД.

4.3.1. Определение характеристик устройства хранения данных

Для получения объективных характеристик производительности подсистемы ввода-вывода была проведена серия тестов с использованием специализированного инструмента `fio`. Данный инструмент позволяет моделировать сценарии работы с диском, приближённые к реальной нагрузке систем управления базами данных.

Следует отметить, что облачный провайдер не предоставляет информацию о модели и технических характеристиках используемого устройства хранения данных. Пользователь получает лишь сведения о лимитах производительности ввода-вывода (IOPS) и типе носителя (SSD), что ограничивает возможности для детального анализа физических параметров диска.

В частности, для оценки параметров устройства хранения данных использовалась следующая команда:

Листинг 4.3 — Команда тестирования диска с помощью утилиты fio

```
1 fio --randrepeat=1 --ioengine=libaio --direct=1 --
    gtod_reduce=1 --name=test --filename=test --bs=4k --
```

```
iodepth=64 --size=1G --readwrite=randrw --rwmixread
=75 > disk_benchmark.txt
```

Параметры использования ключей `fio` следующие:

- `--randrepeat=1` — обеспечивается воспроизводимость тестирования за счёт фиксированного генератора случайных чисел;
- `--ioengine=libaio` — используется асинхронный движок ввода-вывода для максимально полного использования возможностей современной операционной системы;
- `--direct=1` — операции выполняются в режиме прямого доступа, минуя кэш операционной системы;
- `--gtod_reduce=1` — минимизация накладных расходов на получение меток времени;
- `--name=test` — имя задания;
- `--filename=test` — путь к тестовому файлу;
- `--bs=4k` — размер блока ввода-вывода составляет 4 КБ, что соответствует типичному размеру страницы в PostgreSQL;
- `--iodepth=64` — глубина очереди запросов 64, что имитирует параллельную обработку нескольких запросов;
- `--size=1G` — размер тестируемого файла составляет 1 ГБ;
- `--readwrite=randrw` — моделируются смешанные случайные чтения и записи;
- `--rwmixread=75` — 75% операций приходится на чтение, остальные 25% на запись;
- `> disk_benchmark.txt` — результаты тестирования выводятся в файл для последующего анализа.

Полученные данные были использованы для калибровки и построения модели оценки нагрузки на подсистему ввода-вывода при работе СУБД PostgreSQL.

Листинг 4.4 — Результаты тестирования устройства ввода-вывода

```
test: (g=0): rw=randrw, bs=(R) 4096B-4096B, (W) 4096B-4096B, (T) 4096B-4096B, ioengine=libaio, iodepth=64
fio-3.36
Starting 1 process
test: Laying out IO file (1 file / 1024MiB)

test: (groupid=0, jobs=1): err= 0: pid=81186: Wed May 21 20:03:26 2025
  read: IOPS=1425, BW=5702KiB/s (5839kB/s)(768MiB/137838msec)
   bw (  KiB/s): min= 4840, max= 9168, per=100.00%, avg=5705.48, stdev=277.34, samples=275
   iops        : min= 1210, max= 2292, avg=1426.37, stdev=69.34, samples=275
  write: IOPS=476, BW=1905KiB/s (1951kB/s)(256MiB/137838msec); 0 zone resets
   bw (  KiB/s): min= 1616, max= 3048, per=100.00%, avg=1905.37, stdev=114.02, samples=275
   iops        : min=  404, max=  762, avg=476.34, stdev=28.50, samples=275
  cpu          : usr=0.41%, sys=2.00%, ctx=216175, majf=0, minf=13
  IO depths    : 1=0.1%, 2=0.1%, 4=0.1%, 8=0.1%, 16=0.1%, 32=0.1%, >=64=100.0%
     submit    : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 32=0.0%, 64=0.0%, >=64=0.0%
     complete  : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 32=0.0%, 64=0.1%, >=64=0.0%
     issued rwts: total=196498,65646,0,0 short=0,0,0,0 dropped=0,0,0,0
     latency   : target=0, window=0, percentile=100.00%, depth=64

```

```
Run status group 0 (all jobs):
   READ: bw=5702KiB/s (5839kB/s), 5702KiB/s-5702KiB/s
      (5839kB/s-5839kB/s), io=768MiB (805MB), run
      =137838-137838msec
  WRITE: bw=1905KiB/s (1951kB/s), 1905KiB/s-1905KiB/s
     (1951kB/s-1951kB/s), io=256MiB (269MB), run
     =137838-137838msec

Disk stats (read/write):
  vda: ios=196751/65774, sectors=1583944/527632, merge
     =0/176, ticks=5283763/3509394, in_queue=8793157,
     util=97.27%
```

4.4 Сравнение результатов предсказания с реальными замерами

В результате работы программы-предсказателя нагрузки был получен следующий файл:

Листинг 4.5 — Результат предсказания для example1

`example1-predict-result.yaml`

```
postgres_disk_io_prediction:
  database: my_database
  files:
  - access_pattern:
      io_scenarios:
      - cache_hit_ratio: 0.0
        read_ops: 3
        read_throughput_mb_s: 80
        write_ops: 2
        write_throughput_mb_s: 40
      - ...
      total_operations: 50
    estimated_size_mb: 3906.25
```

```
    name: users
    type: heap
  - access_pattern:
      io_scenarios:
      - cache_hit_ratio: 0.0
        read_ops: 3
        read_throughput_mb_s: 80
        write_ops: 2
        write_throughput_mb_s: 40
      - ...
      total_operations: 50
    estimated_size_mb: 172334.562
    name: users_toast
    type: toast
  - access_pattern:
      io_scenarios:
      - cache_hit_ratio: 0.0
        read_ops: 3
        read_throughput_mb_s: 80
        write_ops: 2
        write_throughput_mb_s: 40
      - ...
      total_operations: 50
    estimated_size_mb: 3814.703
    name: users_toast_index
    type: toast
  - access_pattern:
      io_scenarios:
      - cache_hit_ratio: 0.0
        read_ops: 3
        read_throughput_mb_s: 80
        write_ops: 2
        write_throughput_mb_s: 40
      - ...
```

```
48        total_operations: 50
49      estimated_size_mb: 9375.0
50      name: orders
51      type: heap
52    - access_pattern:
53        io_scenarios:
54        - cache_hit_ratio: 0.0
55          read_ops: 3
56          read_throughput_mb_s: 80
57          write_ops: 2
58          write_throughput_mb_s: 40
59        - ...
60        total_operations: 50
61      estimated_size_mb: 137867.648
62      name: orders_toast
63      type: toast
64    - access_pattern:
65        io_scenarios:
66        - cache_hit_ratio: 0.0
67          read_ops: 3
68          read_throughput_mb_s: 80
69          write_ops: 2
70          write_throughput_mb_s: 40
71        - ...
72        total_operations: 50
73      estimated_size_mb: 3051.758
74      name: orders_toast_index
75      type: toast
76    tablespace: pg_default
```

Согласно представленным данным, ожидаемая нагрузка на файловую систему при нулевом попадании в кэш составляет порядка 160 MB/s. Использование значения нулевого попадания в кэш обусловлено характером тестируемой нагрузки — в данном случае имитируется случайное чтение с нормально распределённым смещением относитель-

но среднего значения, что минимизирует эффективность кэширования чтения.

Листинг 4.6 — Результат замеров для example1

example1-result.yaml

```
#             PostgreSQL

## Δ PostgreSQL I/O:

###   pg_stat_io Δ(       /):
  backend|relation|main          | Δread: 1749 Δ
    write: 0
  checkpointer|relation|main     | Δread: 0 Δ     write
    : 300
  background|relation|fsm        | Δread: 50 Δ    write
    : 0

###   pg_statio_user_tables Δ( disk read/cache hit):
  users                          | Δdisk read: 450 Δ
    cache hit: 749550
  orders                         | Δdisk read: 1350 Δ
    cache hit: 1198650 Δ


    (/proc/42564/io):  (             )
rchar:     : 4465126804 (4258.28 MB total,
  446512680.40/sec)
wchar:     : 3276873890 (3125.07 MB total,
  327687389.00/sec)
syscr:     :     420186 (410.34 KB total, 42018.60/sec
  )
syscw:     :     400371 (390.99 KB total, 40037.10/sec
  )

```

```
21   : 443 MB (44.30 MB/s)
22     /: 391420  (39142.00 IOPS)
23
24 ##           - (iostat):
25
26 : sdaΔ
27
28  iostat   :
29 r/s:     0.00 -> 288.00
30 w/s:     0.00 -> 288.00
31 rKB/s:   0.00 -> 46080.00
32 wKB/s:   0.00 -> 46080.00
33 await:   0.64 -> 3.10
```

Анализируя результаты замеров, можно отметить, что фактическое попадание в кэш чтения составило 424 564 из 4 401 240 677 обращений, то есть менее 0,01%, что подтверждает допустимость предположения о пренебрежимо малом влиянии кэша на итоговую нагрузку. При таких условиях предсказатель оценивает нагрузку максимально близко к реальной ситуации дисковых обращений.

Необходимо отметить, что заявленные характеристики используемого в эксперименте дискового устройства составляют не более 45 MB/s по последовательному чтению, в то время как рассчитанная программой-предсказателем нагрузка существенно превышает этот лимит. На практике получение подобной нагрузки для данной системы невозможно, что априори указывает на потенциальное возникновение нерасчётных задержек и деградации производительности. Таким образом, полученный прогноз позволяет на этапе планирования эксплуатации заблаговременно выявлять сценарии некорректного проектирования нагрузки на систему ввода-вывода и принимать управляющие решения на стадии внедрения.

Для подтверждения корректности прогноза была проведена экспериментальная оценка: время выполнения всей нагрузки (50 запросов — интенсивность 5 RPS в течение 10 секунд) зафиксировано при помощи скрипта мониторинга. Фактически полученное время вы-

полнения оказалось сопоставимо с теоретически расчётным, учитывая ограничение по пропускной способности диска, что подтверждает адекватность прогноза, произведённого разработанным средством.

Таким образом, проведённое сравнение демонстрирует практическую ценность применения программного средства для оценки планируемой нагрузки: несмотря на синтетическую природу примера, подход оказывается применимым для заблаговременного предотвращения возникновения ситуаций перегрузки системы ввода-вывода СУБД PostgreSQL.

4.5 Анализ точности предсказаний

Для объективной оценки точности работы разработанного программного средства был проведён ряд экспериментов с различными сценариями нагрузки на систему ввода-вывода СУБД PostgreSQL. В каждом сценарии параметрически задавались типы запросов, их интенсивность, распределение обращений к данным, а также характер возвращаемых данных. Ключевой особенностью моделирования служило жёсткое ограничение — аксимальная пропускная способность исследуемого дискового накопителя составляет 45 МБ/с. Предсказанные значения, превышающие данный предел, аппроксимировались до максимально возможного значения с целью сохранения правдоподобия прогноза.

Результаты сравнения прогнозируемых и эмпирически замеренных значений приведены в таблице 5. В таблице отображены прогнозные значения, скорректированные с учётом физического предела диска, а также соответствующие им реальные показатели, полученные при тестировании; дополнительно рассчитана относительная ошибка прогноза.

Таблица 5 – Сравнение расчётной и реальной нагрузки на систему ввода-вывода (максимальная пропускная способность 45 МБ/с)

Тестовый пример	**Корректированная нагрузка, МБ/с**	**Факт, МБ/с**	**Относительная ошибка, %**
example1	160	208	30,0
example2	40	32	20,0
example3	38	32	15,8
example4	28	39	39,3

4.5.1. Методика приведения расчётных значений к фактическим ограничениям

В рамках экспериментов основная задача заключалась в сопоставлении прогнозируемых нагрузок с фактическими возможностями оборудования — максимальная пропускная способность тестируемого диска составляла 45 МБ/с. Для этого расчётные значения, получаемые на этапе моделирования, сопоставлялись с эмпирическими результатами, полученными при выполнении серии однородных запросов к СУБД.

В каждом сценарии проводилась следующая процедура. Формировалась серия из $rps \times 10$ однотипных запросов, которые, согласно теоретической модели, должны были завершаться за 10 секунд (исходя из предполагаемой интенсивности и объёма обработки данных). В ходе экспериментального замера фиксировалось реальное время, затраченное на выполнение всей серии запросов, которое в ряде случаев превышало расчётное значение из-за ограничения пропускной способности.

Для корректировки прогноза вводился поправочный коэффициент k, определяемый следующим образом:

$$k = \frac{t_{\text{факт}}}{t_{\text{модель}}} \tag{1}$$

где $t_{\text{факт}}$ — фактическое время исполнения серии запросов, $t_{\text{модель}} = 10$ секунд — ожидаемое время по модели.

Далее, с целью приведения моделируемой нагрузки к фактиче-

ским ограничениям аппаратуры, использовалась корректировка нагрузки следующим образом:

$$L_{\text{скорр}} = L_{\text{расчёт}} \times k \tag{2}$$

где $L_{\text{скорр}}$ — скорректированное значение нагрузки, отражающее реальную пропускную способность с учётом аппаратных ограничений. Таким образом, реальная нагрузка увеличивалась на поправочный коэффициент, отражающий замедление выполнения запросов из-за ограничений дисковой подсистемы. Для всех сценариев, в которых изначальный расчёт перегружал систему (значения превышали 45 МБ/с), такой подход позволил объективно привести полученные данные в соответствие с максимальными возможностями оборудования и обеспечить корректность сравнения расчётных и эмпирических результатов.

4.6 Анализ ошибок и отклонений

В результате проведённого тестирования разработанного средства оценки планируемой нагрузки на систему ввода-вывода СУБД PostgreSQL были выявлены существенные отклонения прогнозируемых значений от реальных показателей. Данную особенность можно объяснить наличием множества факторов, не учтённых в модели, что является ожидаемым исходом при работе с комплексными системами ввода-вывода и многопараметрическими нагрузками.

Следует отметить, что данные нюансы и возможные источники отклонений подробно рассматривались в первых главах данной работы, где была обоснована методология построения модели оценки нагрузки и определены её ограничения. Такое поведение системы, а также отмеченные отклонения существенно не противоречат поставленным целям исследования и рассматриваются как приемлемые в рамках текущего этапа разработки.

Особое внимание заслуживает поведение системы в моменты максимальных нагрузок на диск. В указанные периоды наблюдалась стабильно высокая загрузка накопителя, сопровождавшаяся ощути-

мым снижением производительности. Интересно, что именно в эти моменты предсказательная модель демонстрировала завышенные значения планируемой нагрузки, превосходящие фактические возможности дисковой подсистемы. Данный феномен свидетельствует о том, что прогнозирование нагрузки может выявлять потенциальные «узкие места» в инфраструктуре ввода-вывода, что является важным аспектом для последующей оптимизации и масштабирования системы.

4.7 Обсуждение ограничений текущей реализации

Текущая версия программы прогнозирования нагрузки на диск учитывает только обращения к основным файлам таблицы, индексам, TOAST и сопутствующим вспомогательным файлам, основываясь на предполагаемом числе чтений и записей к ним. Однако реализация не учитывает целый ряд важных источников нагрузки, таких как операции UPDATE и DELETE, а также связанные с ними процессы вакуумирования и autovacuum. Помимо этого, в расчетах не моделируется вклад операций логирования (например, дополнительных обращений к WAL), влияние которых может быть заметным при интенсивной работе базы данных.

Ещё одним существенным ограничением является то, что модель работает с обобщенными параметрами кэширования (cache hit ratio) и не принимает во внимание фактическую конфигурацию буферов, показатели системы хранения и механизмы управления памятью PostgreSQL. В реальных условиях существенно влияют дополнительные слои кэширования на стороне операционной системы, особенности реализации драйверов и контроллеров, нагрузка от конкурирующих приложений, а также алгоритмы распределения ввода-вывода на уровне железа. Всё это может приводить к существенным расхождениям между расчетными и фактическими значениями нагрузки.

4.8 Выводы по результатам апробации

В ходе апробации программы были выявлены случаи, когда рассчитанные значения нагрузки на дисковую подсистему заметно отличались от фактических данных, получаемых в реальных условиях

эксплуатации PostgreSQL. Наиболее заметные расхождения наблюдаются при интенсивных смешанных нагрузках, а также при активной работе внутренних механизмов СУБД (например, autovacuum), которые не учтены в текущей реализации.

Тем не менее, несмотря на эту погрешность, полученные результаты имеют практическую ценность: модель позволяет получить ориентировочное представление о возможной нагрузке на диск в зависимости от выбранных сценариев работы с данными и уровней кэширования. Использование подобных прогнозов может быть полезным на ранних стадиях проектирования системы, при сравнительной оценке разных вариантов размещения данных или при предварительной оценке потенциальных проблем с производительностью. В дальнейшем точность прогнозов можно повысить за счёт расширения модели, более детального учета всех механизмов работы PostgreSQL и настройки параметров согласно реальным характеристикам оборудования.

ЗАКЛЮЧЕНИЕ

В ходе данной работы была спроектирована и реализована программа для прогнозирования нагрузки на диск со стороны системы управления базами данных PostgreSQL с целью последующего анализа производительности и обеспечения обоснованного выбора аппаратных и программных решений для хранения и работы с большими объемами данных. В процессе выполнения работы были проанализированы особенности организации файловой структуры PostgreSQL, изучены принципы функционирования системы хранения данных, механизмы обращения к файлам и их влияние на нагрузку ввода-вывода.

Целью разработки являлось создание инструмента, который позволял бы на основании характеристик целевой базы данных (размера файлов, предполагаемой активности, паттернов доступа) оценить потенциальную нагрузку на диск, представить её в разрезе различных вариантов использования кэша и параметров работы PostgreSQL, а также дать возможность разработчикам и администраторам баз данных принимать решения о масштабировании, резервировании ресурсов или оптимизации работы СУБД.

В результате, была реализована программа, формирующая детализированные прогнозы по каждому файлу хранения (основные файлы таблиц, индексы, TOAST, вспомогательные файлы), с учётом предполагаемого числа операций чтения и записи, сценариев уровня попадания в кэш (cache hit ratio) и оценивающая I/O-нагрузку по таким показателям, как количество операций (iops) и пропускная способность (throughput) для различных режимов работы. Программа генерирует структурированный и наглядный результат, удобный для последующего анализа и интеграции в документы проектирования или технические отчёты.

Практическая ценность работы Разработанный инструмент предоставляет удобный способ быстрой оценки дисковой нагрузки при проектировании архитектуры информационной системы на базе PostgreSQL. Программа позволяет без необходимости проведения трудоёмких полноценных нагрузочных испытаний получить первые ориентировочные цифры: объём ожидаемых обращений к файловой системе, про-

пускную способность, зависимость этих величин от параметров кэширования. Это существенно снижает риски при проектировании, упрощает выбор аппаратных решений (тип и объём дисков, организация хранения, резервы под разрастание БД) и формирует базу для последующего более точного тестирования и калибровки. Полученные с помощью данного инструмента оценки могут использоваться для подготовки технических заданий, согласования бюджета на инфраструктуру, сравнения разных вариантов партицирования или индексирования, а также для планирования процедур обслуживания и резервного копирования.

Особенно эффективным представляется применение такой программы на этапах пилотирования проектов, где реальных данных еще недостаточно, а сценарии обращения к БД можно только приблизительно оценить. В таких ситуациях верификация проектных решений с помощью табличной "прогнозной" модели оказывается не только оправданной, но и позволяет избежать выбора заведомо неэффективных стратегий работы с СУБД.

Ограничения и риски При всей полезности предложенного решения необходимо честно отметить ряд существенных ограничений, которые были подробно рассмотрены в соответствующем разделе. Наиболее значимым из них на текущем этапе развития программы является упрощение самой модели нагрузки: не учитываются операции UPDATE и DELETE, а также связанные с ними активности autovacuum и процессов обслуживания таблиц (reindex, анализ, переупаковка и пр.). Помимо этого, в расчетах полностью отсутствует влияние операций логирования, которые в PostgreSQL всегда сопровождают запись в таблицы и могут составлять значительную часть I/O-нагрузки.

Еще одной из сложностей является влияние механизмов кэширования как на уровне PostgreSQL, так и со стороны операционной системы и аппаратных буферов контроллеров и дисков: реальная нагрузка на устройство может быть существенно ниже или выше расчетной в зависимости от особенностей рабочей среды, типа используемой СОСД или гипервизора, физических характеристик используемых носителей и др. В ходе апробации отмечено, что при ряде те-

стов расхождение между прогнозной и измеренной нагрузкой может быть весьма значительным — это связано как с невозможностью точно угадать реальную структуру обращений к данным, так и с поведением внутренних оптимизаторов PostgreSQL, а также непредсказуемым влиянием фоновых процессов.

Вместе с тем автор считает, что предлагаемый подход сохраняет свою ценность, так как получаемые прогнозные данные позволяют выстраивать работу с учетом заведомых рисков: если по расчету нагрузка подходит вплотную к предельным значениям устройств или инфраструктуры, это сигнал о необходимости закладывать дополнительные резервы, либо пересматривать и оптимизировать архитектуру информационной системы до начала эксплуатации и насыщения реальными данными.

Основные результаты и перспективы развития По итогам работы можно выделить основное достижение — разработан удобный в использовании и расширяемый инструмент для прогнозирования дисковой нагрузки, наглядно отображающий вклад каждого компонента хранилища и различных сценариев работы. Программа полностью отделена от специфики аппаратной платформы либо специализированных настроек и может быть использована как отправная точка в различных проектах, с возможностью адаптации под конкретные требования бизнеса и инфраструктуры.

Программа построена на гибкой модели ввода исходных данных и поддерживает расширение спектра анализируемых файлов и сценариев работы; результаты выводятся в формате, пригодном для коммуникации между разработчиками, архитекторами и обслуживающим персоналом.

Важное направление для дальнейшего развития — реализация более точной модели, учитывающей операции обновления и удаления, корректно моделирующей нагрузки от autovacuum и VACUUM FULL, анализирующей влияние WAL (журнала предзаписи) и фоновых процессов планирования, а также интеграция результатов реальных нагрузочных тестов для калибровки коэффициентов и уточнения входных данных. Следующим логичным шагом могла бы стать интегра-

ция с мониторинговыми инструментами, что позволит автоматически сравнивать прогноз и реальные метрики, на лету корректируя модель под конкретную производственную систему.

Кроме этого, в перспективе рассматривается возможность автоматизированного формирования рекомендаций по оптимизации конфигурации PostgreSQL на основании полученных прогнозов, а также генерация предложений по поэтапному переходу на масштабируемые решения (например, sharding, распределенное хранение и др.) при превышении расчетной нагрузки.

Заключительное слово В заключение стоит подчеркнуть, что поставленная задача создания инструмента для прогноза нагрузки на диск в рамках эксплуатации PostgreSQL актуальна как для организаций, разворачивающих новые масштабные ИС с нуля, так и для случаев миграции или реорганизации уже существующих проектов. Предложенная модель не подменяет собой полноценных нагрузочных тестов и не претендует на абсолютную точность, но позволяет существенно повысить осознанность при проектировании, выявить “узкие” места задолго до появления производственных проблем и сэкономить ресурсы на этапе экспериментов и проектирования.

В ходе выполнения данной работы сформирован задел для дальнейших исследований в области автоматизации процессов проектирования и сопровождения баз данных, повышения прозрачности и воспроизводимости процессов оценки нагрузки и в целом снижения издержек на поддержку и масштабирование корпоративных информационных систем.

В заключение отметим, что как для теоретического, так и для прикладного анализа работы PostgreSQL и её взаимодействия с файловой системой важно не только наличие инструментов прогноза, но и практический опыт их применения на реальных данных. Проведённая работа подтверждает, что даже простые прогнозные модели позволяют заранее выявить уязвимости и тем самым существенно сократить как время на внедрение новых решений, так и затраты на их эксплуатацию.

Таким образом, поставленные в работе задачи можно считать

успешно решёнными, а результаты – полезными для последующего применения и развития как в рамках исследовательских проектов, так и в промышленной эксплуатации сложных баз данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Смелянский П. Л.* Эволюция вычислительной инфраструктуры // Вестник Московского университета. Серия 15: Вычислительная математика и кибернетика. — 2024. — № 4. — С. 190—233. — DOI: 10.55959/MSU/0137-0782-15-2024-47-4-190-233. — Lomonosov Computational Mathematics and Cybernetics Journal.
2. *Han J.*, *Choi Y.* Performance Characteristics of PostgreSQL and MariaDB on NVMeVirt // arXiv preprint. — 2024. — arXiv: 2411.10005. — URL: https://arxiv.org/abs/2411.10005.
3. *Zhang N.*, *Tatemura J.*, *Patel J. M.* Towards Cost-Effective Storage Provisioning for DBMSs // arXiv preprint. — 2012. — arXiv: 1201.0226. — URL: https://arxiv.org/abs/1201.0226.
4. *Sardana S.* Reducing Aurora PostgreSQL storage I/O costs : тех. отч. / AWS Database Blog. — 2019.
5. *Вершинин Е. В.*, *Сафин П. Р.* Обзор методов оптимизации базы данных в СУБД PostgreSQL // Наука, техника и образование. — 2023. — 3(43). — URL: https://nto-journal.ru/catalog/informacionnye-texnologii/892/.
6. *Datmt*. Tuning Tips to Maximize Postgres Performance. — 2024. — URL: https://datmt.com/databases/relational-databases/tuning-tips-to-maximize-postgres-performance/ ; Accessed: 2025-05-23.
7. Какие компетенции ждёт региональный рынок ИТ-специалистов? Разработка и апробация технологии анализа актуальных вакансий / А. Е. Шухман [и др.] // Высшее образование в России. — 2022. — Т. 31, № 8/9. — С. 137—153. — ISSN 0869-3617. — DOI: 10.31992/0869-3617-2022-31-8-9-137-153. — URL: http://vovr.elpub.ru ; Vysshee obrazovanie v Rossii = Higher Education in Russia.
8. *Hellerstein J. M.*, *Stonebraker M.*, *Hamilton J.* Architecture of a Database System // Foundations and Trends in Databases. — 2007. — Vol. 1, no. 2. — P. 141–259. — DOI: 10.1561/1900000002.

9. *Малых В. Л.*, *Калинин А. Н.* Миграция с Oracle на PostgreSQL // Программные системы: теория и приложения. — 2022. — Т. 13, № 4. — DOI: 10.25209/2079-3316-2022-13-4-77-91.

10. *Лазарев Н. А.*, *Борисенко О. Д.* Построение требований и архитектуры облачного оркестратора платформенных сервисов // Труды Института системного программирования РАН. — 2022. — Т. 34, № 4. — С. 211—228. — DOI: 10.15514/ISPRAS-2022-34(4)-15. — URL: https://doi.org/10.15514/ISPRAS-2022-34(4)-15.

11. *Борисова А. А.*, *Борисенко О. Д.* Исследование методов построения облачных платформенных сервисов и реализаций стандарта TOSCA // Труды Института системного программирования РАН. — 2022. — Т. 34, № 5. — С. 143—162. — DOI: 10.15514/ISPRAS-2022-34(5)-9.

12. *Татарникова Т. М.* Кластеризация данных на лету для СУБД PostgreSQL / Программные продукты и системы. — 2023. — Т. 36, № 2. — С. 196—201. — DOI: 10.15827/0236-235X.142.196-201. — URL: https://doi.org/10.15827/0236-235X.142.196-201.

13. Cardinality Estimation in DBMS: A Comprehensive Benchmark Evaluation / Y. Han [и др.] // arXiv preprint. — 2021. — arXiv: 2109.05877 [cs.DB]. — URL: https://arxiv.org/abs/2109.05877.

14. Разработка системы мониторинга для серверного приложения / А. Ю. А. Альфара [и др.] // International Journal of Open Information Technologies. — 2023. — Т. 11, № 8. — ISSN 2307-8162.

15. *Полтавцева М. А.*, *Зегжда Д. П.* Моделирование информационных процессов систем управления большими данными для решения задач кибербезопасности // Программные продукты и системы. — 2024. — Т. 37, № 1. — С. 54—61. — DOI: 10.15827/0236-235X.142.054-061. — URL: https://doi.org/10.15827/0236-235X.142.054-061.

16. *Varlamov D. A.*, *Nikiforov I. V.*, *Ustinov S. M.* Algorithm for monitoring and improving the stability of the IT infrastructure based on availability and reliability metrics // Software of Computer, Telecommunications

and Control Systems. — 2023. — DOI: 10.18721/JCSTCS.17203. — URL: https://doi.org/10.18721/JCSTCS.17203 ; UDC 004.052.3.

17. *Zaghloul M Salem M A.-E. A.* A new framework based on features modeling and ensemble learning to predict query performance // PLOS ONE. — 2021. — Vol. 1, no. 1. — DOI: 10.1371/journal.pone.0300197.

18. *Chen J. H.*, *Asch S. M.* Machine learning for electronic health records-based analytics: A review // Jornal of biomedical informatics. — 2019. — Т. 87. — С. 112—127. — DOI: 10.1016/j.jbi.2018.12.003.

19. *Sun J.*, *Wong A.* Predictive analytics and artificial intelligence in healthcare: Promise and pitfalls // Healthcare Analytics. — 2023. — Т. 3. — С. 100100. — DOI: 10.1016/j.health.2023.100100.

20. Dynamic Optimization of Storage Systems Using Reinforcement Learning Techniques / C. Cheng [и др.] // arXiv preprint. — 2025. — arXiv: 2501.00068 [cs.DC]. — URL: https://arxiv.org/abs/2501.00068.

21. *Lapin I. A.*, *Sabinin O. Y.* Development of the system of automatic generation of database model on the basis of the task text in natural language // Software of Computer, Telecommunications and Control Systems. — 2023. — DOI: 10.18721/JCSTCS.17309. — URL: https://doi.org/10.18721/JCSTCS.17309 ; UDC 004.652:004.912.

22. *Ozkaya U.*, *Oztaysi B.*, *Baysan C.* Deep learning for smart manufacturing: Methods and applications // Journal of Manufacturing Systems. — 2020. — Т. 56. — С. 361—368. — DOI: 10.1016/j.jmsy.2020.07.006.

23. *The PostgreSQL Global Development Group*. PostgreSQL 17. Официальная документация. — 2024. — URL: https://www.postgresql.org/docs/17/index.html ; Дата обращения: 23.05.2025.

24. *Фисков А.* Внутренности PostgreSQL: журнал предзаписи (WAL) // Записки программиста. — 2025. — URL: https://eax.me/postgresql-wal/.

25. *Опарина Т. М.* Протоколирование действий пользователей в базе данных // Математические структуры и моделирование. — 2024. — 2 (70). — С. 100—104. — DOI: 10.24147/2222-8772.2024.2.100-

104. — URL: https://cyberleninka.ru/article/n/organizatsiya-zhurnala-operatsiy-s-bazoy-dannyh-v-sisteme-postgresql.

26. *The PostgreSQL Global Development Group*. PostgreSQL. Хранение данных: TOAST. — 2024. — URL: https://www.postgresql.org/docs/current/storage-toast.html ; Дата обращения: 23.05.2025.

ПРИЛОЖЕНИЕ А. Настройки тестовой среды

```
# -----------------------------
# PostgreSQL configuration file
# -----------------------------
# This file consists of lines of the form:

#   name = value
#
# (The "=" is optional.)  Whitespace may be used.  Comments are introduced with
# "#" anywhere on a line.  The complete list of parameter names and allowed
# values can be found in the PostgreSQL documentation.
#
# The commented-out settings shown in this file represent the default values.
# Re-commenting a setting is NOT sufficient to revert it to the default value;
# you need to reload the server.
#
# This file is read on server startup and when the server receives a SIGHUP
# signal.  If you edit the file on a running system, you have to SIGHUP the
# server for the changes to take effect, run "pg_ctl reload", or execute
# "SELECT pg_reload_conf()".  Some parameters, which are marked below,

data_directory = '/var/lib/postgresql/17/main'      # use data in another directory
hba_file = '/etc/postgresql/17/main/pg_hba.conf'    # host-based authentication file
ident_file = '/etc/postgresql/17/main/pg_ident.conf'    # ident configuration file
external_pid_file = '/var/run/postgresql/17-main.pid'           # write an extra PID file
port = 5432             # (change requires restart)
max_connections = 100           # (change requires restart)
unix_socket_directories = '/var/run/postgresql' # comma-separated list of directories

ssl = on
ssl_cert_file = '/etc/ssl/certs/ssl-cert-snakeoil.pem'
ssl_key_file = '/etc/ssl/private/ssl-cert-snakeoil.key'

shared_buffers = 128MB          # min 128kB
#huge_pages = try           # on, off, or try
                    # (change requires restart)
#huge_page_size = 0         # zero for system default
                    # (change requires restart)
#temp_buffers = 8MB         # min 800kB
#max_prepared_transactions = 0      # zero disables the feature
                    # (change requires restart)
```

```
# Caution: it is not advisable to set max_prepared_transactions nonzero unless
# you actively intend to use prepared transactions.
#work_mem = 4MB             # min 64kB
#hash_mem_multiplier = 2.0      # 1-1000.0 multiplier on hash table work_mem
#maintenance_work_mem = 64MB        # min 64kB
#autovacuum_work_mem = -1       # min 64kB, or -1 to use maintenance_work_mem
#logical_decoding_work_mem = 64MB   # min 64kB
#max_stack_depth = 2MB          # min 100kB
#shared_memory_type = mmap      # the default is the first option
                    # supported by the operating system:
                    #   mmap
                    #   sysv
                    #   windows
                    # (change requires restart)
dynamic_shared_memory_type = posix  # the default is usually the first option
                    # supported by the operating system:
                    #   posix
                    #   sysv
                    #   windows
                    #   mmap
                    # (change requires restart)
#min_dynamic_shared_memory = 0MB    # (change requires restart)
#vacuum_buffer_usage_limit = 2MB    # size of vacuum and analyze buffer access
    strategy ring;
                    # 0 to disable vacuum buffer access strategy;
                    # range 128kB to 16GB

# SLRU buffers (change requires restart)
#commit_timestamp_buffers = 0       # memory for pg_commit_ts (0 = auto)
#multixact_offset_buffers = 16      # memory for pg_multixact/offsets
#multixact_member_buffers = 32      # memory for pg_multixact/members
#notify_buffers = 16            # memory for pg_notify
#serializable_buffers = 32      # memory for pg_serial
#subtransaction_buffers = 0     # memory for pg_subtrans (0 = auto)
#transaction_buffers = 0        # memory for pg_xact (0 = auto)

# - Disk -

#temp_file_limit = -1           # limits per-process temp file space
                    # in kilobytes, or -1 for no limit

#max_notify_queue_pages = 1048576   # limits the number of SLRU pages
    allocated
                    # for NOTIFY / LISTEN queue

# - Kernel Resources -

#max_files_per_process = 1000       # min 64
```

```
                    # (change requires restart)

# - Cost-Based Vacuum Delay -

#vacuum_cost_delay = 0          # 0-100 milliseconds (0 disables)
#vacuum_cost_page_hit = 1       # 0-10000 credits
#vacuum_cost_page_miss = 2      # 0-10000 credits
#vacuum_cost_page_dirty = 20        # 0-10000 credits
#vacuum_cost_limit = 200        # 1-10000 credits

# - Background Writer -

#bgwriter_delay = 200ms         # 10-10000ms between rounds
#bgwriter_lru_maxpages = 100        # max buffers written/round, 0 disables
#bgwriter_lru_multiplier = 2.0      # 0-10.0 multiplier on buffers scanned/
    round
#bgwriter_flush_after = 512kB       # measured in pages, 0 disables

# - Asynchronous Behavior -
# WRITE-AHEAD LOG
#------------------------------------------------------------------------------

# - Settings -

#wal_level = replica            # minimal, replica, or logical
                    # (change requires restart)
#fsync = on             # flush data to disk for crash safety
                    # (turning this off can cause
                    # unrecoverable data corruption)
#synchronous_commit = on        # synchronization level;
                    # off, local, remote_write, remote_apply, or on
#wal_sync_method = fsync        # the default is the first option
                    # supported by the operating system:
                    #   open_datasync
                    #   fdatasync (default on Linux and FreeBSD)
                    #   fsync
                    #   fsync_writethrough
                    #   open_sync
#full_page_writes = on          # recover from partial page writes
#wal_log_hints = off            # also do full page writes of non-critical
    updates
                    # (change requires restart)
#wal_compression = off          # enables compression of full-page writes;
                    # off, pglz, lz4, zstd, or on
#wal_init_zero = on         # zero-fill new WAL files
#wal_recycle = on           # recycle WAL files
#wal_buffers = -1           # min 32kB, -1 sets based on shared_buffers
```

```
                    # (change requires restart)
#wal_writer_delay = 200ms       # 1-10000 milliseconds
#wal_writer_flush_after = 1MB       # measured in pages, 0 disables
#wal_skip_threshold = 2MB

#commit_delay = 0           # range 0-100000, in microseconds
#commit_siblings = 5            # range 1-1000

# - Checkpoints -

#checkpoint_timeout = 5min      # range 30s-1d
#checkpoint_completion_target = 0.9 # checkpoint target duration, 0.0 - 1.0
#checkpoint_flush_after = 256kB     # measured in pages, 0 disables
#checkpoint_warning = 30s       # 0 disables
max_wal_size = 1GB
min_wal_size = 80MB
#cpu_tuple_cost = 0.01          # same scale as above
#cpu_index_tuple_cost = 0.005       # same scale as above
#cpu_operator_cost = 0.0025     # same scale as above
#parallel_setup_cost = 1000.0       # same scale as above
#parallel_tuple_cost = 0.1      # same scale as above
#min_parallel_table_scan_size = 8MB
#min_parallel_index_scan_size = 512kB
#effective_cache_size = 4GB
log_line_prefix = '%m [%p] %q%u@%d '        # special values:
log_timezone = 'Etc/UTC'

cluster_name = '17/main'            # added to process titles if nonempty

# AUTOVACUUM
#------------------------------------------------------------------------------

#autovacuum = on            # Enable autovacuum subprocess?  'on'
                    # requires track_counts to also be on.
#autovacuum_max_workers = 3     # max number of autovacuum subprocesses
                    # (change requires restart)
#autovacuum_naptime = 1min      # time between autovacuum runs
#autovacuum_vacuum_threshold = 50   # min number of row updates before
                    # vacuum
#autovacuum_vacuum_insert_threshold = 1000  # min number of row inserts
                        # before vacuum; -1 disables insert
                        # vacuums
#autovacuum_analyze_threshold = 50  # min number of row updates before
                    # analyze
#autovacuum_vacuum_scale_factor = 0.2   # fraction of table size before vacuum
#autovacuum_vacuum_insert_scale_factor = 0.2    # fraction of inserts over
    table
```

```
                        # size before insert vacuum
#autovacuum_analyze_scale_factor = 0.1  # fraction of table size before
    analyze
#autovacuum_freeze_max_age = 200000000  # maximum XID age before forced vacuum
                    # (change requires restart)
#autovacuum_multixact_freeze_max_age = 400000000    # maximum multixact age
                            # before forced vacuum
                            # (change requires restart)
#autovacuum_vacuum_cost_delay = 2ms # default vacuum cost delay for
                    # autovacuum, in milliseconds;
                    # -1 means use vacuum_cost_delay
#autovacuum_vacuum_cost_limit = -1  # default vacuum cost limit for
                    # autovacuum, -1 means use
                    # vacuum_cost_limit

datestyle = 'iso, mdy'
timezone = 'Etc/UTC'
lc_messages = 'C.UTF-8'         # locale for system error message
lc_monetary = 'C.UTF-8'         # locale for monetary formatting
lc_numeric = 'C.UTF-8'          # locale for number formatting
lc_time = 'C.UTF-8'         # locale for time formatting
default_text_search_config = 'pg_catalog.english'

include_dir = 'conf.d'          # include files ending in '.conf' from
```

Листинг 1 – Файл конфигурации `postgresql.conf`

ПРИЛОЖЕНИЕ Б. Конфигурация для экспериментов

Б.1 `example1.yaml`

```yaml
load_prediction_config:
  tables:
    - name: users
      ddl: |
        CREATE TABLE users (
          id SERIAL PRIMARY KEY,
          email TEXT,
          interests TEXT[],
          profile JSONB
        );
      row_count: 500000

    - name: orders
      ddl: |
        CREATE TABLE orders (
          id SERIAL PRIMARY KEY,
```

```
17          user_id INT,
18          amount NUMERIC,
19          status TEXT
20        );
21      row_count: 1200000
22
23  load_generator:
24    query: |
25      SELECT u.email, o.amount
26      FROM users u
27      JOIN orders o ON u.id = o.user_id
28      WHERE u.interests @> ARRAY['tech'] AND o.status = 'paid';
29    rps: 5
30
31  postgresql_config:
32    ...
33
34  environment:
35    cpu_cores: 2
36    ram_gb: 2
37    storage_type: "ssd"
```

Листинг 2 – Конфигурация для эксперимента `example1.yaml`

Б.2 `example2.yaml`

```
1  load_prediction_config:
2    tables:
3      - name: sensor_data
4        ddl: |
5          CREATE TABLE sensor_data (
6            id BIGSERIAL PRIMARY KEY,
7            sensor_id INT,
8            temperature FLOAT,
9            reading_time TIMESTAMPTZ
10         );
11       row_count: 100000000
12
13   load_generator:
14     query: |
15       SELECT sensor_id, AVG(temperature)
16       FROM sensor_data
17       WHERE reading_time BETWEEN now() - interval '1 day' AND now()
18       GROUP BY sensor_id;
19     rps: 3
20
21   postgresql_config:
22     ...
```

```yaml

  environment:
    cpu_cores: 2
    ram_gb: 2
    storage_type: "ssd"
```

Листинг 3 – Конфигурация для эксперимента `example2.yaml`

Б.3 `example3.yaml`

```yaml
load_prediction_config:
  tables:
    - name: accounts
      ddl: |
        CREATE TABLE accounts (
          id SERIAL PRIMARY KEY,
          login TEXT UNIQUE,
          password_hash TEXT,
          email TEXT
        );
      row_count: 200000

  load_generator:
    query: |
      SELECT id FROM accounts WHERE login = 'test_user';
    rps: 1

  postgresql_config:
    ...

  environment:
    cpu_cores: 2
    ram_gb: 2
    storage_type: "ssd"
```

Листинг 4 – Конфигурация для эксперимента `example3.yaml`

Б.4 `example4.yaml`

```yaml
load_prediction_config:
  tables:
    - name: logs
      ddl: |
        CREATE TABLE logs (
          id BIGSERIAL PRIMARY KEY,
          timestamp TIMESTAMPTZ NOT NULL DEFAULT now(),
          message TEXT,
```

```
9           metadata JSONB
10        );
11      row_count: 0  #       -
12
13  load_generator:
14    query: |
15      INSERT INTO logs (message, metadata)
16      VALUES ('Something happened', '{"level": "info", "component": "backend"}
          ');
17    rps: 2
18
19  postgresql_config:
20    ...
21
22  environment:
23    cpu_cores: 2
24    ram_gb: 2
25    storage_type: "ssd"
```

Листинг 5 – Конфигурация для эксперимента `example4.yaml`

ПРИЛОЖЕНИЕ В. Часть алгоритма прогнозирования

```
1  import math
2  import re
3  from .constants import PAGE_SIZE, TOAST_THRESHOLD, FILL_FACTOR,
     DEFAULT_FILL_FACTOR, FSM_SIZE, VM_SIZE
4  from .file_model import create_file_entry
5  from .ddl_parser import extract_columns_and_indexes_from_ddl
6
7  def estimate_column_size_pg(col_type):
8      c = col_type.lower()
9      if 'int' in c and not 'big' in c:
10         return 4
11     if 'bigint' in c:
12         return 8
13     if 'smallint' in c:
14         return 2
15     if 'serial' in c or 'bigserial' in c:
16         return 4 if 'big' not in c else 8
17     if 'numeric' in c or 'decimal' in c:
18         m = re.search(r'$(\d+),\s*(\d+)$', c)
19         return 16 if m else 32
20     if 'timestamp' in c or 'date' in c or 'time' in c:
21         return 8
22     if 'bool' in c:
23         return 1
```

```
    if 'varchar' in c or 'char' in c:
        m = re.search(r'$(\d+)$', c)
        return int(m.group(1)) if m else 64
    if 'text' in c or 'json' in c or 'jsonb' in c or 'xml' in c or 'bytea' in c:
        #          TOASTable,            ,
        return 3000
    return 32  # default size

def estimate_index_row_size(cols, index_type):
    if index_type in ['btree', 'gist', 'spgist', 'brin']:
        #        -          + oid + overhead,          ;
        return sum(estimate_column_size_pg(col_type) for _, col_type in cols) + 8
    if index_type == 'gin':
        # GIN -    --             ,
        return 64  #                      jsonb gin
    return 32

def estimate_table_storage(name, ddl, row_count, rps, is_joined):
    cols, indexes, params = extract_columns_and_indexes_from_ddl(ddl)
    row_overhead = 24  # header + xmin/xmax, oid, t_cid    .
    row_size = row_overhead
    toastable_cols = []
    for cname, col_type in cols:
        col_size = estimate_column_size_pg(col_type)
        row_size += min(col_size, TOAST_THRESHOLD)
        #        TOAST
        if col_size >= TOAST_THRESHOLD or any(t in col_type.lower() for t in ['text', 'json', 'xml', 'bytea']):
            toastable_cols.append((cname, col_type, col_size))

    files = {}

    # HEAP/TABLE
    table_fill_factor = params.get('fillfactor', FILL_FACTOR)
    rows_per_page = max(1, math.floor(PAGE_SIZE * (table_fill_factor / 100) / row_size))
    total_table_pages = math.ceil(row_count / rows_per_page)
    files[name] = create_file_entry(
        name, total_table_pages * PAGE_SIZE, "Heap access (join read)" if is_joined else "Heap (unused)", rps
    )
    files[f"{name}_fsm"] = create_file_entry(f"{name}_fsm", FSM_SIZE(total_table_pages), "Free Space Map", 0)
    files[f"{name}_vm"] = create_file_entry(f"{name}_vm", VM_SIZE(total_table_pages), "Visibility Map", 0)
```

```
    files[f"{name}_init"] = create_file_entry(f"{name}_init", PAGE_SIZE, "Init fork", 0)

    # TOAST   -    (        )
    if toastable_cols:
        #                  ,  -
        avg_toasted = sum((c[2]-TOAST_THRESHOLD) for c in toastable_cols if c[2] > TOAST_THRESHOLD) / max(len(toastable_cols),1)
        toast_row_size = max(avg_toasted, 1000) + row_overhead
        toast_rows = row_count * len([c for c in toastable_cols if c[2] > TOAST_THRESHOLD])
        toast_pages = math.ceil(toast_rows * toast_row_size / (PAGE_SIZE * (table_fill_factor/100)))
        files[f"{name}_toast"] = create_file_entry(
            f"{name}_toast", toast_pages * PAGE_SIZE, "TOAST data", rps
        )
        # TOAST    -      btree
        index_rows = toast_rows
        index_row_size = 24  # btree on oid+chunk_id
        index_pages = math.ceil(index_rows * index_row_size / (PAGE_SIZE * (DEFAULT_FILL_FACTOR/100)))
        files[f"{name}_toast_index"] = create_file_entry(
            f"{name}_toast_index", index_pages * PAGE_SIZE, "TOAST index (btree)", rps
        )
        files[f"{name}_toast_fsm"] = create_file_entry(f"{name}_toast_fsm", FSM_SIZE(toast_pages), "TOAST FSM", 0)
        files[f"{name}_toast_vm"] = create_file_entry(f"{name}_toast_vm", VM_SIZE(toast_pages), "TOAST VM", 0)

    #       (,           )
    for idx in indexes:
        idx_name = idx['name']
        idx_cols = [col for col in cols if col[0] in idx['columns']]
        idx_type = idx.get('type', 'btree').lower()
        unique_str = " (unique)" if idx.get('unique', False) else ""
        ind_fill_factor = idx.get('fillfactor', DEFAULT_FILL_FACTOR)
        index_row_size = estimate_index_row_size(idx_cols, idx_type)
        index_rows = row_count  #             partial,     -      partial,            idx
        #               (, GIN   json)                  ,
        index_pages = math.ceil(index_rows * index_row_size / (PAGE_SIZE * (ind_fill_factor / 100)))
        files[f"{name}_{idx_name}"] = create_file_entry(
            f"{name}_{idx_name}", index_pages * PAGE_SIZE, f"Index {idx_type}{unique_str}", rps
        )
```

```
100         files[f"{name}_{idx_name}_fsm"] = create_file_entry(f"{name}_{idx_name
            }_fsm", FSM_SIZE(index_pages), f"Index {idx_type} FSM", 0)
101         files[f"{name}_{idx_name}_vm"] = create_file_entry(f"{name}_{idx_name}
            _vm", VM_SIZE(index_pages), f"Index {idx_type} VM", 0)
102         files[f"{name}_{idx_name}_init"] = create_file_entry(f"{name}_{
            idx_name}_init", PAGE_SIZE, f"Index {idx_type} INIT", 0)
103
104     return files
```

Листинг 6 – Часть алгоритма прогнозирования, файл `estimators.py`

ПРИЛОЖЕНИЕ Г. Репозиторий с исходным кодом проекта

Исходный код программной части дипломного проекта доступен в репозитории GitHub по ссылке: `https://github.com/ShchA317/diploma`.